

Nivel Experto

Nivel Experto: Programación Funcional con fpdart y Result Pattern

La Programación Funcional (PF) no es solo una moda: es una filosofía que te obliga a **modelar el fallo como parte del dominio**. En aplicaciones modernas donde la red es inestable, los APIs cambian, y los usuarios esperan experiencias fluidas, manejar errores explícitamente no es opcional — es arquitectura.

1. Fundamentos: ¿Por qué Programación Funcional en Dart?

Dart ya tiene null safety, pero eso solo resuelve el problema de “valor ausente”. No resuelve el problema de “operación fallida”. Cuando llamas a una API, existen tres estados:

1. **Éxito**: Tienes un resultado válido
2. **Error de dominio**: La operación falló por una regla de negocio (ej: “saldo insuficiente”)
3. **Error técnico**: La red cayó, el servidor respondió 500, timeout

La PF te obliga a modelar estos tres estados de forma **exhaustiva**. El compilador te obliga a manejar cada caso. No hay `try-catch` olvidados, no hay `null` inesperado.

El Problema con try-catch tradicional

```
// ❌ Enfoque tradicional: excepciones como control flow
Future<User> getUser() async {
  try {
    final response = await api.get('/user');
    return User.fromJson(response);
  } catch (e) {
    // ¿Qué tipo de error? ¿Red? ¿Parseo? ¿Servidor?
    throw Exception('Error: $e'); // Información perdida
  }
}
```

El Enfoque Funcional

```
// ✅ Enfoque funcional: el tipo dice TODO
Future<Either<Failure, User>> getUser() async {
  // Either<Left(Error), Right(Success)>
  // Left traditionally contiene el error
  // Right traditionally contiene el éxito
}
```

La firma del método te dice todo lo que necesitas saber: puede fallar, y el fallo tiene un tipo concreto (`Failure`).

2. Dartz vs fpdart: La Evolución del Ecosistema

Históricamente, `dartz` fue la librería estándar para FP en Dart. Sin embargo, el ecosistema ha evolucionado y `fpdart` se ha convertido en la elección moderna.

Comparativa Técnica

Aspecto	dartz	fpdart
Mantenimiento	Abandonado desde 2021	Activo (última versión 2024)
Sintaxis	Estilo Haskell puro	Idioms de Dart modernos
Integración Null Safety	Incompleta	Total
Extensiones de IDE	Pocas	Excelentes (tipos bien definidos)
Tamaño bundle	~200KB	~150KB (tree shaking)
Documentación	Escasa	Completa con ejemplos
API de Streams	Básica	Avanzada (parsers, validations)

Ejemplo Práctico: Transformación de Datos

Con dartz:

```
import 'package:dartz/dartz.dart';

Either<Failure, int> result = Right(10);
final mapped = result.map((r) => r * 2);
// works, pero las extensiones son limitadas
```

Con fpdart:

```
import 'package:fpdart/fpdart.dart';

Either<Failure, int> result = Right(10);

// map: transformar el valor contenido
final mapped = result.map((r) => r * 2); // Right(20)

// flatMap: encadenar operaciones que retornan Either
final chained = result.flatMap((r) => Right(r * 2));

// getOrElse: obtener valor con fallback
final value = result.getOrElse(() => 0); // 10

// fold: procesar ambos casos
result.match(
  (failure) => print('Error: $failure'),
  (value) => print('Valor: $value'),
);

// match: igual que fold pero más conciso (alias)
result.match(
  (failure) => print('Error: $failure'),
  (value) => print('Valor: $value'),
);
```

3. Either: El Corazón de la Gestión de Errores

3.1 La Semántica de Left y Right

Existe una convención en FP: - **Left**: Representa el valor “incorrecto” o error - **Right**: Representa el valor “correcto” o éxito

```
// Convenciones en fpdart
Right<String, int>(42) // ✅ Éxito: el int 42
Left<String, int>('error') // ❌ Fallo: el string 'error'
```

3.2 Creación de Either

```
// Constructor directo
Right<Failure, User>(user);
Left<Failure, User>(ServerFailure('500'));

// From nullable (convierte null a Failure)
Either<Failure, User> fromNullable(
  User? user,
  Failure Function() ifNull,
) => user != null ? Right(user) : Left(ifNull());

// From Future (convierte Future a Either)
Future<Either<Failure, User>> fromPromise(
  Future<User> promise,
  Failure Function(Object error) onError,
) async {
  try {
    return Right(await promise);
  } catch (e) {
    return Left(onError(e));
  }
}
```

4. Option/Maybe: El Homólogo de Null Safety

En países donde null safety no existe (Dart pre-2.12), `Option` era esencial. Hoy, tiene un propósito diferente: representar ausencia intencional de valor que no es un error.

Cuándo usar Option vs Null

Escenario	Solución
Usuario no ha iniciado sesión	<code>User? user = null</code> (natural)
Búsqueda no encontró resultados	<code>Option<User> user</code> (ausencia intencional)
Configuración opcional	<code>Option<Theme> theme</code>
Campo que puede no existir en JSON	<code>int? age</code> (null safety nativo)

Ejemplo con Option

```
import 'package:fpdart/fpdart.dart';

Option<int> findUserId(String email) {
  final users = [{email: 'a@test.com', 'id': 1}];

  final found = users.firstWhere(
    (u) => u['email'] == email,
    orElse: () => {},
  );

  // Map a Option: si no existe, None
  return found.isEmpty
    ? const None()
    : Option<int>.fromNullable(found['id']);
}

// Uso
findUserId('a@test.com').match(
  () => print('No encontrado'), // None
  (id) => print('ID: $id'),     // Some(id)
);
```

5. Task y TaskEither: Asincronía Funcional

5.1 El Problema con Future

`Future` no tiene contexto de error tipado. Además, no puedes componerlas fácilmente:

```
// ❌ Future: composición difícil
Future<User> getUser() async {
  final token = await getToken(); // puede fallar silenciosamente
  final user = await api.getUser(token);
  return user;
}
```

5.2 Task: Future con Contexto

`Task<A>` es un wrapper alrededor de `Future<A>` que permite composición:

```
import 'package:fpdart/fpdart.dart';

// Task<A> = () -> Future<A>
// Es una función perezosa que cuando la ejecutas retorna un Future

Task<User> getUserTask = Task(() async {
  final token = await getToken();
  return await api.getUser(token);
});

// Ejecutar cuando quieras
Future<User> user = getUserTask.run();
```

5.3 TaskEither: El Poder Combinado

`TaskEither<Failure, A> = Task<Either<Failure, A>>`

Esto es **transformador de efectos**: manejar asincronía Y errores tipados en una sola estructura.

```
TaskEither<Failure, User> getUserTask(String id) {
    return TaskEither.tryCatch(
        () => api.getUser(id),
        (error, stackTrace) => ServerFailure(error.toString()),
    );
}

// Encadenar operaciones (no hay try-catch en ninguna parte)
final result = await getUserTask('123')
    .map((user) => user.name.toUpperCase())
    .flatMap((name) => TaskEither.of(Greeting(name)))
    .run();
```

Ejemplo Completo: Repository con TaskEither

```
abstract class UserRepository {
    TaskEither<Failure, User> getUser(String id);
    TaskEither<Failure, List<User>> getUsers();
}

class UserRepositoryImpl implements UserRepository {
    final UserRemoteDataSource remoteDataSource;

    UserRepositoryImpl(this.remoteDataSource);

    @override
    TaskEither<Failure, User> getUser(String id) {
        return TaskEither.tryCatch(
            () => remoteDataSource.getUser(id),
            (error, stackTrace) {
                if (error is SocketException) {
                    return NetworkFailure('Sin conexión');
                }
                if (error is TimeoutException) {
                    return NetworkFailure('Tiempo de espera agotado');
                }
                return ServerFailure(error.toString());
            },
        );
    }

    @override
    TaskEither<Failure, List<User>> getUsers() {
        return TaskEither.tryCatch(
            () => remoteDataSource.getUsers(),
            (error, stackTrace) => ServerFailure(error.toString()),
        ).map((users) => users.where((u) => u.isActive).toList());
    }
}
```

6. El Result Pattern (Sin Dependencias)

Si quieres un **Domain puro** sin librerías externas, Dart 3+ con Sealed Classes es suficiente.

6.1 Implementación Básica

```
// lib/core/error/result.dart

sealed class Result<S, E> {
  const Result();
}

final class Success<S, E> extends Result<S, E> {
  final S value;
  const Success(this.value);
}

final class Failure<S, E> extends Result<S, E> {
  final E error;
  const Failure(this.error);
}
```

6.2 Extensiones de Utilidad

```
extension ResultExtension<S, E> on Result<S, E> {
    bool get isSuccess => this is Success<S, E>;
    bool get isFailure => this is Failure<S, E>;

    S getOrThrow() {
        return switch (this) {
            Success(value: final v) => v,
            Failure(error: final e) => throw Exception('Unexpected failure:
                $e'),
        };
    }

    S getOrElse(S defaultValue) {
        return switch (this) {
            Success(value: final v) => v,
            Failure() => defaultValue,
        };
    }

    Result<R, E> map<R>(R Function(S) f) {
        return switch (this) {
            Success(value: final v) => Success(f(v)),
            Failure(error: final e) => Failure(e),
        };
    }

    Result<S, R> mapError<R>(R Function(E) f) {
        return switch (this) {
            Success(value: final v) => Success(v),
            Failure(error: final e) => Failure(f(e)),
        };
    }

    Future<Result<R, E>> mapFuture<R>(Future<R> Function(S) f) async {
        return switch (this) {
            Success(value: final v) => Success(await f(v)),
            Failure(error: final e) => Failure(e),
        };
    }
}
```

6.3 Uso Exhaustive (El Poder del Compilador)

```
Future<Result<User, Failure>> getUser(String id) async { ... }

void handle() {
    final result = await getUser('123');

    // El compilador fuerza manejar TODOS los casos
    switch (result) {
        case Success(value: final user):
            print('Usuario: ${user.name}');
        case Failure(error: final err):
            print('Error: ${err.message}');
            // Posiblemente reintentar, mostrar UI, etc.
    }
}
```


7. Fallback Chains: Operadores de Rescue

A veces quieres intentar una operación, y si falla, intentar otra. Esto se llama “fallback chain” o “rescue”.

Con Either

```
// Intentar API primaria, luego secundaria
Future<Either<Failure, Data>> getData() async {
  final primary = await primarySource.getData();

  return primary.match(
    (failure) async {
      // Fallback: intentar fuente secundaria
      final secondary = await secondarySource.getData();
      return secondary.match(
        (e) => Failure('Both failed: $e'), // Todas las opciones
        (data) => Right(data),
      );
    },
    (data) => Right(data),
  );
}
```

Con fpdart: pipe y chain

```
// Usando extension methods
final result = await primarySource.getData()
  .orElse(() => secondarySource.getData())
  .orElse(() => cachedSource.getData())
  .run();
```

8. Errores Comunes y Cómo Evitarlos

Error 1: map vs flatMap

```
Either<Failure, int> divide(int a, int b) {
  if (b == 0) return Left(DivisionFailure());
  return Right(a ~/ b);
}

// ❌ map cuando necesitas retornar Either
final result = divide(10, 2).map((r) => divide(r, 2));
// Error: flatMap needed! map expects (int) -> int, not (int) -> Either

// ✅ flatMap para encadenar operaciones que pueden fallar
final result = divide(10, 2).flatMap((r) => divide(r, 2));
```

Error 2: Pérdida de Tipos en fold

```
// ❌ fold sin especificar tipos puede perder tipos
result.match(
  (e) => print(e), // e puede ser Any
  (v) => print(v), // v puede ser Any
);

// ✅ Especifica los tipos
result.fold<Widget>(
  (failure) => ErrorWidget(failure.message),
  (user) => UserWidget(user),
);
```

Error 3: Olvidar ejecutar TaskEither

```
// ❌ Crear pero no ejecutar
final task = getUserTask('123');
// ERROR: task es TaskEither, no User!

// ✅ Ejecutar con .run()
final result = await getUserTask('123').run();
```

9. Integración Completa: Clean Architecture con fpdart

Domain Layer: UseCase

```
// lib/features/users/domain/usecases/get_user.dart

@lazySingleton
class GetUser extends UseCase<User, GetUserParams> {
  final UserRepository repository;

  GetUser(this.repository);

  @override
  Future<Either<Failure, User>> call(GetUserParams params) async {
    return repository.getUser(params.id);
  }
}

class GetUserParams {
  final String id;
  const GetUserParams(this.id);
}
```

Data Layer: Repository Implementation

```
// lib/features/users/data/repositories/user_repository_impl.dart

@LazySingleton(as: UserRepository)
class UserRepositoryImpl implements UserRepository {
  final UserRemoteDataSource remoteDataSource;
  final UserLocalDataSource localDataSource;

  UserRepositoryImpl(this.remoteDataSource, this.localDataSource);

  @override
  Future<Either<Failure, User>> getUser(String id) async {
    return TaskEither.tryCatch(
      () => remoteDataSource.getUser(id),
      (error, stackTrace) {
        if (error is SocketException) {
          return NetworkFailure('Sin conexión a internet');
        }
        if (error is UnauthorizedException) {
          return AuthFailure('Sesión expirada');
        }
        return ServerFailure(error.toString());
      },
    ).flatMap((user) {
      return TaskEither.tryCatch(
        () => localDataSource.cacheUser(user),
        (_, __) => CacheFailure('Error guardando en cache'),
      );
    }).run();
  }
}
```

Presentation Layer: Cubit

```
// lib/features/users/presentation/cubit/user_cubit.dart

@injectable
class UserCubit extends Cubit<UserState> {
  final GetUser getUser;

  UserCubit(this.getUser) : super(UserInitial());

  Future<void> loadUser(String id) async {
    emit(UserLoading());

    final result = await getUser(GetUserParams(id));

    result.match(
      (failure) => emit(UserError(failure.message)),
      (user) => emit(UserLoaded(user)),
    );
  }
}
```

10. Comparativa: ¿Cuándo Usar Qué?

Escenario	Herramienta Recomendada
Proyecto nuevo, mediano-grande	fpdart (TaskEither)
Proyecto pequeño, quiere mínimo código	Result Pattern (Sealed Classes)
Equipo sin experiencia en FP	Result Pattern
Necesitas transformar muchos datos	fpdart (extensiones ricas)
Firebase/API con muchos errores	fpdart (TaskEither composition)
Solo necesitas Either básico	dartz (si ya lo tienes)

11. Recomendaciones de Rendimiento

Tree Shaking

fpdart está diseñado para tree shaking:

```
# pubspec.yaml - asegurate de no importar todo
dependencies:
  fpdart: ^1.2.0
# No importes package:fpdart/fpdart.dart
# Mejor: import solo lo que necesitas
```

Bundle Size Impact

Librería	Tamaño Minificado
dartz	~180 KB
fpdart	~120 KB
freezed + json_serializable	~250 KB

12. Recetas Rápidas

Validación de Input con Either

```
Either<ValidationFailure, User> validateUser(String name, String
    email) {
    if (name.isEmpty) {
        return Left(ValidationFailure('Nombre requerido'));
    }
    if (!email.contains('@')) {
        return Left(ValidationFailure('Email inválido'));
    }
    return Right(User(name, email));
}
```

Reintentos con TaskEither

```
TaskEither<Failure, T> withRetry(
    TaskEither<Failure, T> task, {
    int maxRetries = 3,
}) {
    return task.orElse((failure) {
        // Lógica de retry
        return withRetry(task, maxRetries: maxRetries - 1);
    });
}
```

Timeout

```
TaskEither<Failure, T> withTimeout(
    TaskEither<Failure, T> task, {
    Duration timeout = const Duration(seconds: 10),
}) {
    return TaskEither.tryCatch(
        () => task.run().timeout(timeout),
        (error, _) => TimeoutFailure(),
    );
}
```

13. Testing con fpdart y Result Pattern

El testing es donde FP brilla: los tipos explícitos hacen que los tests sean más predecibles y los mocks más controlados.

13.1 Configuración de Testing

```
# pubspec.yaml (dev)
dev_dependencies:
  mocktail: ^1.0.3
  flutter_test:
    sdk: flutter
```

13.2 Unit Testing de UseCases con Either

Setup básico:

```
// test/features/users/domain/usecases/get_user_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/users/domain/repositories/user_repository.dart';
import 'package:my_app/features/users/domain/usecases/get_user.dart';

// Mock del repository
class MockUserRepository extends Mock implements UserRepository {}

void main() {
  late GetUser useCase;
  late MockUserRepository mockRepository;

  setUp(() {
    mockRepository = MockUserRepository();
    useCase = GetUser(mockRepository);
  });

  setUpAll(() {
    // Registrar fallback values
    registerFallbackValue(GetUserParams('test'));
  });

  group('GetUser UseCase', () {
    const tUser = User(id: '1', name: 'Test User');
    const tParams = GetUserParams('1');
    const tFailure = ServerFailure('Server error');

    test('debería retornar User cuando el repository succeeds', () {
      async {
        // Arrange
        when(() => mockRepository.getUser(tParams.id))
          .thenAnswer((_) async => const Right(tUser));

        // Act
        final result = await useCase(tParams);

        // Assert
        expect(result, const Right(tUser));
        verify(() => mockRepository.getUser(tParams.id)).called(1);
        verifyNoMoreInteractions(mockRepository);
      });

    test('debería retornar Failure cuando el repository fails', () {
      async {
        // Arrange
        when(() => mockRepository.getUser(tParams.id))
          .thenAnswer((_) async => const Left(tFailure));

        // Act
        final result = await useCase(tParams);

        // Assert
        expect(result, const Left(tFailure));
        verify(() => mockRepository.getUser(tParams.id)).called(1);
      });
    });
  });
}
```

13.3 Testing de TaskEither

```
// test/features/users/data/repositories/user_repository_impl_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/users/data/datasources/user_remote_datasource.dart';
import 'package:my_app/features/users/data/repositories/user_repository_impl.dart';
import 'package:my_app/core/error/failures.dart';

class MockUserRemoteDataSource extends Mock implements
    UserRemoteDataSource {}

void main() {
  late UserRepositoryImpl repository;
  late MockUserRemoteDataSource mockDataSource;

  setUp(() {
    mockDataSource = MockUserRemoteDataSource();
    repository = UserRepositoryImpl(mockDataSource);
  });

  group('getUser TaskEither', () {
    const tUser = User(id: '1', name: 'Test');
    const tFailure = ServerFailure('Error');

    test('debería retornar Right(User) cuando succeeds', () async {
      // Arrange
      when(() => mockDataSource.getUser('1'))
        .thenAnswer((_) async => UserModel(id: '1', name: 'Test'));

      // Act
      final result = await repository.getUser('1').run();

      // Assert
      result.match(
        (failure) => fail('No debería fallar'),
        (user) => expect(user.name, 'Test'),
      );
    });

    test('debería retornar Left(Failure) cuando falla', () async {
      // Arrange
      when(() => mockDataSource.getUser('1'))
        .thenThrow(Exception('Server error'));

      // Act
      final result = await repository.getUser('1').run();

      // Assert
      result.match(
        (failure) => expect(failure, isA<ServerFailure>()),
        (_) => fail('No debería retornar usuario'),
      );
    });

    test('debería mapear error de red a NetworkFailure', () async {
      // Arrange
      when(() => mockDataSource.getUser('1'))
        .thenThrow(SocketException('No internet'));

      // Act
      final result = await repository.getUser('1').run();

      // Assert
      result.match(
        (failure) => expect(failure.message, contains('conexión')),
      );
    });
  });
}
```

```

        (_) => fail('No debería retornar usuario'),
    );
  });
});
}

```

13.4 Testing de Validación con Either

```

// test/core/validators/user_validator_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/validators/user_validator.dart';

void main() {
  late UserValidator validator;

  setUp(() {
    validator = UserValidator();
  });

  group('validateUser', () {
    test('debería retornar Right cuando datos son válidos', () {
      // Act
      final result = validator.validateUser('John', 'john@test.com');

      // Assert
      expect(result.isRight(), true);
    });

    test('debería retornar Left(ValidationFailure) cuando nombre vacío', () {
      // Act
      final result = validator.validateUser('', 'john@test.com');

      // Assert
      result.match(
        (failure) => expect(failure.message, 'Nombre requerido'),
        (_) => fail('No debería ser válido'),
      );
    });

    test('debería retornar Left cuando email inválido', () {
      // Act
      final result = validator.validateUser('John', 'invalid-email');

      // Assert
      result.match(
        (failure) => expect(failure.message, 'Email inválido'),
        (_) => fail('No debería ser válido'),
      );
    });

    test('debería retornar primer error encontrado (fail fast)', () {
      // Arrange: ambos campos inválidos
      final result = validator.validateUser('', '');

      // Assert: solo primer error
      result.match(
        (failure) => expect(failure.message, 'Nombre requerido'),
        (_) => fail('No debería ser válido'),
      );
    });
  });
}

```


13.5 Testing del Result Pattern (Sealed Classes)

```
// test/core/error/result_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/core/error/result.dart';

void main() {
  group('Result Pattern', () {
    test('Success debe contener el valor', () {
      const result = Success<String, String>('data');

      expect(result.isSuccess, true);
      expect(result.isFailure, false);
      expect(result.value, 'data');
    });

    test('Failure debe contener el error', () {
      const result = Failure<String, String>('error');

      expect(result.isSuccess, false);
      expect(result.isFailure, true);
      expect(result.error, 'error');
    });

    test('getOrElse debe retornar valor en Success', () {
      const result = Success<int, String>(42);
      expect(result.getOrElse(0), 42);
    });

    test('getOrElse debe retornar default en Failure', () {
      const result = Failure<int, String>('error');
      expect(result.getOrElse(0), 0);
    });

    test('map debe transformar el valor', () {
      const result = Success<int, String>(10);
      final mapped = result.map((v) => v * 2);

      expect(mapped.value, 20);
    });

    test('map debe pasar el error sin transformar', () {
      const result = Failure<int, String>('error');
      final mapped = result.map((v) => v * 2);

      expect(mapped.error, 'error');
    });

    test('switch exhaustivo debe compilar', () {
      const success = Success<int, String>(42);
      const failure = Failure<int, String>('error');

      void handleResult(Result<int, String> result) {
        switch (result) {
          case Success(value: final v):
            print('Success: $v');
          case Failure(error: final e):
            print('Failure: $e');
        }
      }

      handleResult(success);
      handleResult(failure);
    });
  });
}
```

13.6 Testing de Integración: Repository → UseCase → Cubit

```
// test/features/users/presentation/cubit/user_cubit_test.dart

import 'package:bloc_test/bloc_test.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/users/domain/usecases/get_user.dart';
import 'package:my_app/features/users/presentation/cubit/user_cubit.dart';
import 'package:my_app/core/error/failures.dart';

class MockGetUser extends Mock implements GetUser {}

void main() {
  late UserCubit cubit;
  late MockGetUser mockGetUser;

  setUp(() {
    mockGetUser = MockGetUser();
    cubit = UserCubit(mockGetUser);
  });

  tearDown(() {
    cubit.close();
  });

  setUpAll(() {
    registerFallbackValue(GetUserParams('test'));
  });

  group('UserCubit', () {
    const tUser = User(id: '1', name: 'Test User');
    const tFailure = ServerFailure('Error');

    test('estado inicial debe ser UserInitial', () {
      expect(cubit.state, UserInitial());
    });

    blocTest<UserCubit, UserState>(
      'debería emitir [Loading, Loaded] cuando getUser succeeds',
      build: () {
        when(() => mockGetUser(any()))
          .thenAnswer((_) async => const Right(tUser));
        return UserCubit(mockGetUser);
      },
      act: (cubit) => cubit.loadUser('1'),
      expect: () => [
        UserLoading(),
        const UserLoaded(tUser),
      ],
      verify: (_) {
        verify(() => mockGetUser(const
          GetUserParams('1'))).called(1);
      },
    );

    blocTest<UserCubit, UserState>(
      'debería emitir [Loading, Error] cuando getUser fails',
      build: () {
        when(() => mockGetUser(any()))
          .thenAnswer((_) async => const Left(tFailure));
        return UserCubit(mockGetUser);
      },
      act: (cubit) => cubit.loadUser('1'),
      expect: () => [
        UserLoading(),
        const UserError(tFailure.message),
      ],
    );
  });
}
```

```

    ],
  );
});
}

```

13.7 Errores Comunes en Testing

```

// ❌ Error: No registrar fallback values
setUpAll(() {
  // CRASH: Fallback value not set for GetUserParams
});

// ✅ Solución: Registrar antes de usar
setUpAll(() {
  registerFallbackValue(GetUserParams('fallback'));
});

// ❌ Error: Olvidar tearDown
tearDown(() {
  cubit.close(); // CRASH: Stream not closed
});

// ✅ Solución: Siempre cerrar
tearDown(() {
  cubit.close();
});

// ❌ Error: No mockear todos los métodos necesarios
when(() => mockRepo.getUser('1')).thenAnswer(...);
// CRASH: Unmocked method called

// ✅ Solución: Mockear todo lo que se llama
when(() => mockRepo.getUser(any())).thenAnswer(...);

```

13.8 Tabla de Herramientas de Testing

Escenario	Herramienta Recomendada
Mock de interfaces	<code>mocktail</code> (sin setup)
Mock de classes	<code>mockito</code> (con generation)
Testing de BLoC/Cubit	<code>bloc_test</code>
Testing de streams	<code>test</code> + <code>fake_async</code>
Testing de excepciones	<code>test('throws', ...)</code>

Resumen Ejecutivo

1. **fpdart** es el estándar moderno para FP en Dart:
 - mantenimiento activo, sintaxis idiomatic, excelente integración con null safety
2. **Either** modela operaciones que pueden fallar con tipos explícitos
3. **TaskEither** combina asincronía y manejo de errores en una estructura composable

4. **Result Pattern** con Sealed Classes es una alternativa sin dependencias para proyectos pequeños
5. **El compilador es tu amigo**: úsalo para forzar manejo exhaustivo de errores
6. **flatMap** es esencial para encadenar operaciones que retornan Either
7. La integración con Clean Architecture es natural: Repository retorna Either, UseCase lo pasa, Cubit lo consume con fold

La próxima vez que escribas un `try-catch`, pregúntate: ¿estoy ocultando información del tipo? ¿Debería usar Either?

Recursos Adicionales

- Documentación oficial fpdart
 - FP in Dart - Francesco Gullà
 - Railway Oriented Programming - El concepto fundamental de Either
-

Ver también

- `04-ALMACENAMIENTO-LOCAL` — Persistencia de datos con drift, Hive, SharedPreferences
 - `16-BLOC-CUBIT` — Gestión de estado con BLoC/Cubit
-

En el siguiente módulo

→ `02-di-injectable.md` — Dependency Injection automatizada con Injectable y GetIt

Nivel Experto

Nivel Experto: Automatización de Inyección de Dependencias con Injectable

La Inyección de Dependencias (DI) es el pegamento que mantiene Clean Architecture unido. Sin embargo, en proyectos reales, el archivo `injection_container.dart` se convierte rápidamente en un monstruo de miles de líneas. **Injectable** resuelve esto generando código automáticamente basándose en anotaciones.

1. Fundamentos: De Manual a Automático

1.1 El Problema del DI Manual

Imagina un proyecto con 50+ clases que necesitan inyección:

```
//  injection_container.dart - EL MUNDO SIN INJECTABLE

final sl = GetIt.instance;

Future<void> init() async {
  // Data Sources
  sl.registerLazySingleton<UserRemoteDataSource>(
    () => UserRemoteDataSourceImpl(sl()),
  );
  sl.registerLazySingleton<UserLocalDataSource>(
    () => UserLocalDataSourceImpl(sl()),
  );

  // Repositories
  sl.registerLazySingleton<UserRepository>(
    () => UserRepositoryImpl(sl(), sl()),
  );

  // Use Cases
  sl.registerLazySingleton(() => GetUser(sl()));
  sl.registerLazySingleton(() => SaveUser(sl()));
  sl.registerLazySingleton(() => DeleteUser(sl()));

  // Cubits
  sl.registerFactory(() => UserCubit(sl(), sl(), sl()));

  // External
  sl.registerLazySingleton(() => http.Client());

  // ... 50+ más líneas similares
}
```

Problemas: - Cada nuevo use case = 2+ líneas manual - Errores solo se detectan en runtime - Refactorizaciones = actualizaciones manuales tediosas - Difícil de mantener en equipo

1.2 La Solución: Injectable

```
//  injection_container.dart - CON INJECTABLE

final sl = GetIt.instance;

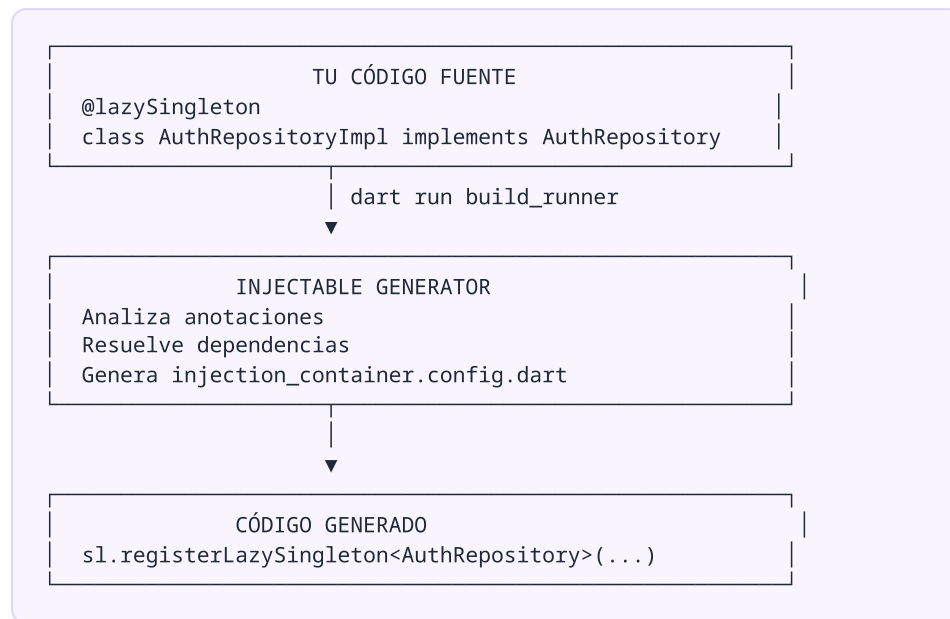
@InjectableInit()
Future<void> init() async => sl.init();

// Las anotaciones generan TODO automáticamente
```

Beneficios: - Nuevas clases = solo agregar `@lazySingleton` - Errores en tiempo de compilación - Soporte para environments - Testing simplificado

2. Arquitectura de Injectable

2.1 Cómo Funciona el Generador



2.2 Flujo de Resolución de Dependencias

```
@LazySingleton(as: UserRepository)
class UserRepositoryImpl
  implements UserRepository {
  final UserRemoteDataSource remoteDataSource;
  final UserLocalDataSource localDataSource;

  UserRepositoryImpl(this.remoteDataSource, this.localDataSource);
}
```

El generador: 1. Ve la anotación `@LazySingleton(as: UserRepository)` 2. Infiere que `UserRepository` es la interfaz 3. Busca constructor y ve: `UserRemoteDataSource`, `UserLocalDataSource` 4. Genera:

```
sl.registerLazySingleton<UserRepository>(
  () => UserRepositoryImpl(
    sl<UserRemoteDataSource>(),
    sl<UserLocalDataSource>(),
  ),
);
```

3. Configuración Completa

3.1 Dependencias Requeridas

```
# pubspec.yaml

dependencies:
  get_it: ^7.6.4
  injectable: ^2.4.1

dev_dependencies:
  injectable_generator: ^2.6.1
  build_runner: ^2.4.8
```

3.2 Configuración de build.yaml

```
# build.yaml - Configuración del generador

targets:
  $default:
    builders:
      injectable_generator:injectable_builder:
        options:
          # Auto-register dependencias comunes
          auto_register: true

          # Buscar en estos directorios
          root_directory:
            path: lib
            generate_on: false

          # Imports que agregar automáticamente
          imports:
            - package:http/http.dart
            - package:shared_preferences/shared_preferences.dart

          # Options de configuración
          synthetic_package: false
          relative_paths: true
```

3.3 Estructura de Carpetas Recomendada

```
lib/
├── core/
│   ├── di/
│   │   ├── injection_container.dart      # Entry point
│   │   ├── injection_container.config.dart # GENERADO
│   │   └── modules/
│   │       ├── external_module.dart      # Librerías externas
│   │       ├── database_module.dart      # DB connections
│   │       └── network_module.dart       # HTTP clients
│   └── environments/
│       ├── dev.env                      # Config dev
│       └── prod.env                     # Config prod
```


4. Anotaciones en Profundidad

4.1 Tipos de Scopes

Anotación	Descripción	Cuándo Usar
<code>@lazySingleton</code>	Una sola instancia, creada lazily	Repos, DataSources, UseCases
<code>@singleton</code>	Una sola instancia, creada inmediatamente	Config, AuthService
<code>@injectable</code>	Nueva instancia cada vez	Cubits, BLoCs, Presenters
<code>@factory</code>	Alias de <code>@injectable</code>	Mismo propósito

4.2 Anotaciones de Clase

```
// DataSource - típicamente singleton
@lazySingleton
class UserRemoteDataSourceImpl implements UserRemoteDataSource {
    final http.Client client;
    UserRemoteDataSourceImpl(this.client);
}
```

```
// Repository con interfaz
@LazySingleton(as: UserRepository)
class UserRepositoryImpl implements UserRepository {
    final UserRemoteDataSource remoteDataSource;
    final UserLocalDataSource localDataSource;
    UserRepositoryImpl(this.remoteDataSource, this.localDataSource);
}
```

```
// UseCase - siempre singleton
@lazySingleton
class GetUser extends UseCase<User, String> {
    final UserRepository repository;
    GetUser(this.repository);
}
```

```
// Cubit/Bloc - factory (nueva instancia por inyección)
@injectable
class UserCubit extends Cubit<UserState> {
    final GetUser getUser;
    UserCubit(this.getUser) : super(UserInitial());
}
```

```
// Screen/Widget - factory también
@injectable
class UserScreen extends StatelessWidget {
    final UserCubit cubit;
    UserScreen(this.cubit);
}
```

4.3 Parámetros de Constructor

A veces necesitas pasar valores que no son injectables:

```
@lazySingleton
class FetchUsers {
  final UserRepository repository;

  FetchUsers(this.repository); // Solo el repository es inyectado

  Future<void> execute({int page = 1}) async { ... }
}
```

```
@injectable
class UserListCubit extends Cubit<UserListState> {
  final FetchUsers fetchUsers;

  UserListCubit(this.fetchUsers); // Injectable + parámetros
  opcionales

  void loadPage(int page) => fetchUsers.execute(page: page);
}
```

5. Módulos: Inyectando Librerías Externas

5.1 Por Qué Son Necesarios

Librerías como `http`, `shared_preferences`, `firebase` no tienen anotaciones. Los módulos le dicen a `Injectable` cómo crear estas instancias.

5.2 Módulo Básico

```
// lib/core/di/modules/external_module.dart

import 'package:http/http.dart' as http;
import 'package:injectable/injectable.dart';

@module
abstract class ExternalModule {
  // Inyección simple
  @lazySingleton
  http.Client get httpClient => http.Client();

  // Con configuración
  @lazySingleton
  http.Client get authenticatedClient {
    final client = http.Client();
    client.headers['Authorization'] = 'Bearer token';
    return client;
  }
}
```

5.3 Módulo Asíncrono con `@preResolve`

Para dependencias que necesitan `await` (`SharedPreferences`, `Firebase`, `Database`):

```
// lib/core/di/modules/preferences_module.dart

import 'package:injectable/injectable.dart';
import 'package:shared_preferences/shared_preferences.dart';

@module
abstract class PreferencesModule {
  // IMPORTANTE: Debe ser top-level function, no arrow function
  @preResolve
  Future<SharedPreferences> get sharedPreferences {
    return SharedPreferences.getInstance();
  }
}
```

Uso:

```
Future<void> main() async {
  await init(); // sl.init() espera a que @preResolve termine
  runApp(MyApp());
}
```

5.4 Módulos con Configuración

```
// lib/core/di/modules/config_module.dart

import 'package:injectable/injectable.dart';
import 'package:flutter/foundation.dart';
import 'package:firebase_core/firebase_core.dart' as firebase;

@module
abstract class ConfigModule {
  @lazySingleton
  String get apiBaseUrl {
    if (kDebugMode) return 'https://dev.api.com';
    return 'https://api.com';
  }

  @preResolve
  @lazySingleton
  Future<firebase.FirebaseOptions> get firebaseOptions {
    if (kDebugMode) {
      return firebase.FirebaseOptions(
        apiKey: 'dev_key',
        projectId: 'dev-project',
      );
    }
    return firebase.FirebaseOptions(
      apiKey: 'prod_key',
      projectId: 'prod-project',
    );
  }
}
```

6. Environments: Desarrollo vs Producción

6.1 Configuración de Environments

```
// lib/core/di/injection_container.dart

@InjectableInit(
  initializerName: 'init', // nombre de la función
  preferRelativeImports: true,
  asExtension: true, // sl.init() en lugar de init(sl)
)
void configureDependencies({required String environment}) {}
```

6.2 Environments en Clases

```
// Solo registrar en desarrollo
@lazySingleton
@Environment(Environment.dev)
class DevToolsService {
  DevToolsService();
}

// Solo registrar en producción
@lazySingleton
@Environment(Environment.prod)
class AnalyticsService {
  AnalyticsService();
}

// Registrar en ambos
@lazySingleton
@Environment(['dev', 'prod'])
class UserService {
  UserService();
}
```

6.3 Environments con Módulos

```
@module
@Environment(Environment.dev)
abstract class DevModule {
  @lazySingleton
  String get apiUrl => 'http://localhost:8080';
}

@module
@Environment(Environment.prod)
abstract class ProdModule {
  @lazySingleton
  String get apiUrl => 'https://api.production.com';
}
```

6.4 Inicialización con Environment

```
Future<void> main() async {  
  const env = String.fromEnvironment('ENV', defaultValue: 'dev');  
  
  await init(environment: env);  
  
  runApp(MyApp());  
}
```

```
# Run en desarrollo  
flutter run --dart-define=ENV=dev  
  
# Run en producción  
flutter run --dart-define=ENV=prod
```

7. Errores Comunes y Debugging

Error 1: Dependencia Circular

```
Error: Generator cannot resolve dependency for 'UserRepository'  
→ 'UserRepository' depends on 'UserRepository'
```

Solución: Una de las clases debe usar `@lazySingleton` vs `@injectable` para romper el ciclo, o refactorizar.

Error 2: No Encontrar la Interfaz

```
Error: No generator registered for 'UserRepository'  
→ Make sure you have @LazySingleton(as: UserRepository)
```

Solución: Asegúrate de usar `as:` para indicar la interfaz.

Error 3: Constructor Privado

```
Error: Cannot instantiate UserRepository  
→ Constructor must be public for injection
```

Solución: Constructor debe ser `const UserRepositoryImpl()` o al menos público.

Error 4: Módulo No Generado

```
Error: Expected a type name for 'ExternalModule'  
→ Make sure you have @module annotation
```

Solución: Asegúrate de que el módulo es `abstract class`.

8. Testing con Injectable

8.1 Reemplazando Dependencias en Tests

```
import 'package:flutter_test/flutter_test.dart';
import 'package:injectable/injectable.dart';

void main() {
  setUpAll(() {
    // Resetear GetIt
    get.reset();

    // Registrar mocks
    get.registerLazySingleton<UserRepository>(
      () => MockUserRepository(),
    );
  });

  test('getUser returns user', () async {
    final useCase = get<GetUser>();
    final result = await useCase('1');

    expect(result.isRight(), true);
  });
}
```

8.2 Testing con Environment

```
void main() {
  setUpAll(() {
    // Inicializar con environment de test
    init(environment: Environment.test);
  });
}
```

8.3 Módulo de Test

```
@module
@Environment(Environment.test)
abstract class TestModule {
  @lazySingleton
  @override
  http.Client get httpClient => MockClient();
}
```

8.4 Testing de Integración Completo

Cuando necesitas testear la inyección completa:

```
// test/injection_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:injectable/injectable.dart';
import 'package:my_app/injection_container.dart' as di;

void main() {
  setUpAll(() async {
    // Inicializar con environment de test
    await di.init(environment: Environment.test);
  });

  tearDownAll(() {
    // Limpiar después de todos los tests
    di.sl.reset();
  });

  group('Dependency Injection', () {
    test('debería resolver UserRepository', () {
      final repo = di.sl<UserRepository>();
      expect(repo, isNotNull);
    });

    test('debería resolver GetUser UseCase', () {
      final useCase = di.sl<GetUser>();
      expect(useCase, isNotNull);
    });

    test('debería resolver UserCubit como factory (nueva instancia)',
        () {
      final cubit1 = di.sl<UserCubit>();
      final cubit2 = di.sl<UserCubit>();

      // Factory crea nuevas instancias
      expect(cubit1, isNot(same(cubit2)));
    });

    test('debería resolver UserRepository como singleton (misma
        instancia)', () {
      final repo1 = di.sl<UserRepository>();
      final repo2 = di.sl<UserRepository>();

      // Singleton usa la misma instancia
      expect(identical(repo1, repo2), true);
    });
  });
}
```

8.5 Reemplazo Dinámico de Dependencias

Para tests específicos sin modificar el código de producción:

```

void main() {
  group('UserRepository Tests', () {
    setUp(() {
      // Resetea y registrar mocks ANTES de cada test
      di.sl.reset();

      // Registrar mock
      di.sl.registerLazySingleton<UserRemoteDataSource>(
        () => MockUserRemoteDataSource(),
      );

      // Registrar implementación real del repositorio
      di.sl.registerLazySingleton<UserRepository>(
        () => UserRepositoryImpl(di.sl()),
      );
    });

    test('getUser usa el mock', () async {
      final mockDataSource = di.sl<UserRemoteDataSource>() as
        MockUserRemoteDataSource;
      when(() => mockDataSource.getUser('1'))
        .thenAnswer((_) async => UserModel(id: '1', name:
          'Mocked'));

      final repo = di.sl<UserRepository>();
      final result = await repo.getUser('1');

      expect(result.isRight(), true);
      verify(() => mockDataSource.getUser('1')).called(1);
    });

    tearDown(() {
      di.sl.reset();
    });
  });
}

```


8.6 Testing con Múltiples Environments

```
void main() {
  group('Environment Tests', () {
    test('dev environment tiene API correcta', () async {
      await di.init(environment: Environment.dev);

      final config = di.sl<AppConfig>();
      expect(config.apiUrl, 'https://dev.api.com');

      di.sl.reset();
    });

    test('prod environment tiene API correcta', () async {
      await di.init(environment: Environment.prod);

      final config = di.sl<AppConfig>();
      expect(config.apiUrl, 'https://api.production.com');

      di.sl.reset();
    });

    test('test environment usa mocks', () async {
      await di.init(environment: Environment.test);

      // En test, API debería ser mock
      final httpClient = di.sl<http.Client>();
      expect(httpClient, isA<MockClient>());

      di.sl.reset();
    });
  });
}
```

8.7 Mocks con mocktail

```
# pubspec.yaml (dev)
dev_dependencies:
  mocktail: ^1.0.3
```

```
// test/mocks/mocktail_mock.dart

import 'package:mocktail/mocktail.dart';
import 'package:my_app/features/users/data/datasources/user_remote_datasource.dart';
import 'package:my_app/features/users/domain/repositories/user_repository.dart';

class MockUserRemoteDataSource extends Mock implements
  UserRemoteDataSource {}

class MockUserRepository extends Mock implements UserRepository {}

// Registrar fallback values para mocktail
class FakeUserParams extends Fake implements UserParams {}

void main() {
  setUpAll(() {
    registerFallbackValue(FakeUserParams());
  });
}
```

8.8 Testing de UseCase con GetIt

```
// test/features/users/domain/usecases/get_user_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:my_app/features/users/domain/repositories/user_repository.dart';
import 'package:my_app/features/users/domain/usecases/get_user.dart';
import 'package:injectable/injectable.dart';
import 'package:my_app/injection_container.dart' as di;

class MockUserRepository extends Mock implements UserRepository {}

void main() {
  late GetUser useCase;
  late MockUserRepository mockRepository;

  setUp(() async {
    // Inicializar DI
    await di.init(environment: Environment.test);

    // Reemplazar dependencia real con mock
    mockRepository = MockUserRepository();
    di.sl.registerLazySingleton<UserRepository>(() =>
      mockRepository);

    // Obtener UseCase ya inyectado
    useCase = di.sl<GetUser>();
  });

  setUpAll(() {
    registerFallbackValue(const GetUserParams('test'));
  });

  tearDown(() {
    di.sl.reset();
  });

  test('debería retornar usuario del repository', () async {
    // Arrange
    const user = User(id: '1', name: 'Test');
    when(() => mockRepository.getUser(any()))
      .thenAnswer(_ async => const Right(user));

    // Act
    final result = await useCase(const GetUserParams('1'));

    // Assert
    expect(result, const Right(user));
    verify(() => mockRepository.getUser('1')).called(1);
  });
}
```

8.9 Testing de Cubit con GetIt

```
// test/features/users/presentation/cubit/user_cubit_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:bloc_test/bloc_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:my_app/features/users/domain/usecases/get_user.dart';
import 'package:my_app/features/users/presentation/cubit/user_cubit.dart';
import 'package:injectable/injectable.dart';
import 'package:my_app/injection_container.dart' as di;

class MockGetUser extends Mock implements GetUser {}

void main() {
  late MockGetUser mockGetUser;

  setUp(() async {
    await di.init(environment: Environment.test);
    mockGetUser = MockGetUser();

    // Reemplazar GetUse con mock
    di.sl.registerLazySingleton<GetUser>(() => mockGetUser);
  });

  setUpAll(() {
    registerFallbackValue(const GetUserParams('test'));
  });

  tearDown(() {
    di.sl.reset();
  });

  blocTest<UserCubit, UserState>(
    'debería emitir [Loading, Loaded] cuando getUser succeeds',
    setUp: () {
      when(() => mockGetUser(any()))
        .thenAnswer((_) async => const Right(User(id: '1', name: 'Test')));
    },
    build: () => di.sl<UserCubit>(),
    act: (cubit) => cubit.loadUser('1'),
    expect: () => [
      UserLoading(),
      const UserLoaded(User(id: '1', name: 'Test')),
    ],
  );

  blocTest<UserCubit, UserState>(
    'debería emitir [Loading, Error] cuando getUser fails',
    setUp: () {
      when(() => mockGetUser(any()))
        .thenAnswer((_) async => const Left(ServerFailure('Error')));
    },
    build: () => di.sl<UserCubit>(),
    act: (cubit) => cubit.loadUser('1'),
    expect: () => [
      UserLoading(),
      const UserError('Error'),
    ],
  );
}
```

8.10 Errores Comunes en Testing con Injectable

```
// ❌ Error 1: Olvidar reset()
void testSinReset() {
    // CRASH: Instance already exists
}

// ✅ Solución
void testConReset() {
    di.sl.reset();
    // Ahora puedes registrar nuevas dependencias
}

// ❌ Error 2: No registrar fallback value
setUp() {
    // CRASH: MissingStubError
    when(() => mockRepo.getUser(any())).thenReturn(...);
});

// ✅ Solución
setUpAll() {
    registerFallbackValue(UserParams('fallback'));
});

// ❌ Error 3: Environment incorrecto
void testWrongEnv() {
    await di.init(environment: Environment.prod); // Usa config de prod
    // CRASH: Test mocks no están registrados
}

// ✅ Solución
void testCorrectEnv() {
    await di.init(environment: Environment.test);
    // Mocks registrados correctamente
}

// ❌ Error 4: No cerrar StreamController en Cubit
@override
Future<void> close() {
    _controller.close(); // Olvidar en producción → leak en test
    return super.close();
}

// ✅ Solución: Siempre cerrar
@override
Future<void> close() {
    _controller.close();
    return super.close();
}
```

8.11 Estructura de Tests con Injectable

```
test/
├── core/
│   └── di/
│       └── injection_test.dart          # Tests de DI
├── features/
│   ├── users/
│   │   ├── domain/
│   │   │   └── usecases/
│   │   │       └── get_user_test.dart  # Tests de UseCase
│   │   └── presentation/
│   │       └── cubit/
│   │           └── user_cubit_test.dart # Tests de Cubit
│   └── products/
│       └── ...
├── mocks/
│   ├── mocktail_mock.dart             #Mocks globales
│   └── fixtures/
│       └── user_fixture.dart           # Datos de prueba
└── helpers/
    └── test_helpers.dart               # Utils para tests
```

8.12 Configuración de test_config.dart

```
// lib/core/di/test_config.dart

import 'package:injectable/injectable.dart';
import 'package:my_app/injection_container.dart' as di;

@module
@Environment(Environment.test)
abstract class TestConfigModule {
  @lazySingleton
  @override
  http.Client get httpClient => MockClient();

  @lazySingleton
  @override
  SharedPreferences get sharedPreferences => MockSharedPreferences();
}

Future<void> setupTestDependencies() async {
  await di.init(environment: Environment.test);
}

void tearDownTestDependencies() {
  di.sl.reset();
}
```

9. Casos Avanzados

9.1 Inyectar Instancia Existente

```
@module
abstract class ExistingModule {
    // Registrar una instancia ya creada
    @lazySingleton
    @override
    UserRepository get userRepository => existingUserRepository;
}
```

9.2 Named Instances

```
// Diferentes implementaciones para mismo tipo
@Named('http')
@lazySingleton
http.Client httpClient() => http.Client();

@Named('http_dio')
@lazySingleton
http.Client dioClient() => Dio();

// Uso
class ApiService {
    final http.Client client;

    // Inject by name
    ApiService(@Named('http_dio') this.client);
}
```

9.3 Qualifiers Personalizados

```
@Qualifier(value: 'cache')
class CacheQualifier {}

@lazySingleton
@CacheQualifier
class CacheService {}

// Uso
class SomeService {
    final CacheService cache;

    SomeService(@CacheQualifier this.cache);
}
```

10. Mejores Prácticas

10.1 Estructura de Módulos

```
lib/core/di/
├── injection_container.dart      # Entry point
├── modules/
│   ├── external_module.dart     # http, shared_preferences
│   ├── database_module.dart     # SQLite, Hive, Isar
│   ├── firebase_module.dart     # Firebase services
│   └── config_module.dart       # App config
└── repositories/
    └── repositories_module.dart # Repository bindings
```

10.2 Reglas de Oro

1. Usa `@lazySingleton` para UseCases, Repos, DataSources - Son stateless y reutilizables
2. Usa `@injectable` para Cubits/BLoCs - Cada screen necesita su propia instancia
3. Un módulo por tipo de dependencia - Separa concerns
4. Ejecuta `build_runner` después de cada cambio - No te olvides
5. Revisa el código generado - Asegúrate de que se ve correcto

10.3 Orden de Inicialización

```
@InjectableInit(
  initializerName: 'init',
  // Ejecutar en orden específico
  moduleName: 'CoreModule',
)
void configureDependencies() {}

// El orden en injection_container.dart importa
@InjectableInit(
  modules: [
    ExternalModule(),      // 1. Primero: dependencias externas
    DatabaseModule(),      // 2. Segundo: bases de datos
    RepositoryModule(),    // 3. Tercero: repos
    UseCaseModule(),       // 4. Cuarto: casos de uso
    CubitModule(),         // 5. Último: presentation
  ],
)
Future<void> configureDependencies() async {}
```

11. Integración con Riverpod (Bonus)

Si usas Riverpod además de Clean Architecture:

```
// Provider que usa GetIt
final userRepositoryProvider = Provider<UserRepository>(
  (ref) => getIt<UserRepository>(),
);
```

12. Recetas Rápidas

Agregar Nueva Feature

1. Crear UseCase con `@lazySingleton`
2. Crear Repository con `@LazySingleton(as: Interface)`
3. Crear Cubit con `@injectable`
4. Ejecutar: `dart run build_runner build --delete-conflicting-outputs`
5. Listo 

Agregar Nueva Dependencia Externa

1. Crear nuevo módulo o agregar a `ExternalModule`
2. Agregar getter con `@lazySingleton`
3. Ejecutar generador

Cambiar Scope de una Clase

```
// Cambiar de singleton a factory
@injectable // Era @lazySingleton
class MyClass {}
```

Resumen Ejecutivo

1. **Injectable + GetIt** es el estándar de DI en Flutter moderno
2. Las anotaciones (`@lazySingleton` , `@injectable` , `@module`) generan código automáticamente
3. **Environments** permiten diferentes configuraciones para dev/prod
4. **Módulos** son esenciales para librerías externas
5. **Testing** se simplifica con posibilidad de reemplazar dependencias
6. **Errores comunes** son principalmente dependencias circulares y falta de generador

Siguiente nivel: Aprende a combinar Injectable con `freezed` y `json_serializable` para un pipeline completo de generación de código.

Recursos Adicionales

- Documentación oficial Injectable
 - Ejemplos de Injectable
 - Talk: Dependency Injection Done Right
-

Ver también

- `14-LOADED` — Patrón Loaded con dependencias y estado inicial
 - `17-CLEAN-ARCHITECTURE` — Arquitectura limpia y separación de capas
-

En el siguiente módulo

→ `03-comunicacion-features.md` — Comunicación entre features con Event Bus

Nivel Experto

Nivel Experto: Comunicación Entre Features en Clean Architecture

En aplicaciones medianas y grandes, las features no existen en aislamiento. Un e-commerce tiene usuarios, productos, carrito, pagos, notificaciones. Cada uno es una feature, pero necesitan comunicarse. El error más común es **acoplar directamente las features**, rompiendo los principios de Clean Architecture. Esta guía profundiza en patrones robustos para lograr comunicación sin acoplamiento.

1. Fundamentos: Por Qué el Acoplamiento es Peligroso

1.1 El Problema del Acoplamiento Directo

Imaginemos:

Feature_A (Presentation) → Feature_B (Data/Repository)

Si `Feature_A` importa directamente el repositorio de `Feature_B`:

```
// ❌ ACOPLAMIENTO
import 'package:features/products/data/repositories/product_repository.dart';

class CartCubit extends Cubit<CartState> {
  final ProductRepository productRepo; // ⚠️ Feature_A conoce Feature_B

  Future<void> checkout() async {
    final products = await productRepo.getProducts(); // 💣 Alto acoplamiento
    // ...
  }
}
```

Consecuencias: - Si `ProductRepository` cambia, `CartCubit` puede romperse - No puedes reutilizar `Feature_A` sin `Feature_B` - Testing se vuelve difícil - Difícil mantener equipos trabajando en paralelo

1.2 El Principio de Dependencia Inversa

Clean Architecture dice: las capas internas no deben conocer las externas. Pero cuando dos features del mismo nivel necesitan comunicarse, aplicamos **Inversión de Dependencias**.

```
Feature_A → Interfaz (abstracta) ← Feature_B
```

Ninguna feature conoce a la otra. Ambas dependen de una abstracción.

2. Patrones de Comunicación

2.1 Patrón 1: Repository Compartido (Shared Domain)

Cuándo usarlo: Cuando múltiples features necesitan acceder a los mismos datos.

Estructura:

```
lib/
├── core/
│   ├── domain/
│   │   └── repositories/
│   │       └── auth_repository.dart # Interfaz única
├── features/
│   ├── auth/
│   │   ├── data/repositories/auth_repository_impl.dart
│   │   └── presentation/
│   ├── perfil/
│   │   └── presentation/ (usa AuthRepository)
│   └── settings/
│       └── presentation/ (usa AuthRepository)
```

Implementación:

```
// lib/core/domain/repositories/auth_repository.dart

abstract class AuthRepository {
  Future<Option<User>> getCurrentUser();
  Future<Either<Failure, User>> signIn(Email email, Password password);
  Future<Either<Failure, void>> signOut();
  Stream<User?> get authStateChanges;
}
```

```
// lib/features/auth/data/repositories/auth_repository_impl.dart

@LazySingleton(as: AuthRepository)
class AuthRepositoryImpl implements AuthRepository {
  final FirebaseAuth _firebaseAuth;

  AuthRepositoryImpl(this._firebaseAuth);

  @override
  Future<Option<User>> getCurrentUser() async {
    final user = _firebaseAuth.currentUser;
    return Option.fromNullable(user);
  }

  // ... implementación
}
```

```
// lib/features/perfil/presentation/cubit/perfil_cubit.dart

@injectable
class PerfilCubit extends Cubit<PerfilState> {
  final AuthRepository authRepository;
  final GetUserProfile getUserProfile;

  PerfilCubit(this.authRepository, this.getUserProfile)
    : super(PerfilInitial());

  Future<void> loadProfile() async {
    // Solo conoce AuthRepository, NO ProductRepository
    final user = await authRepository.getCurrentUser();

    user.match(
      () => emit(PerfilUnauthenticated()),
      (u) async {
        final profile = await getUserProfile(u.id);
        profile.match(
          (f) => emit(PerfilError(f.message)),
          (p) => emit(PerfilLoaded(p)),
        );
      },
    );
  }
}
```

2.2 Patrón 2: Feature Delegation (Mediator)

Cuándo usarlo: Cuando una feature necesita ejecutar una acción de otra sin conocer su implementación.

Estructura conceptual:

```
Feature_A (Checkout)
  ↓ usa
Feature_B UseCase (ClearProductCache)
  ↓ implementa
Feature_B Repository → DataSource
```

Implementación:

```
// lib/features/products/domain/usecases/clear_product_cache.dart

@lazySingleton
class ClearProductCache {
  final ProductLocalDataSource localDataSource;

  ClearProductCache(this.localDataSource);

  Future<void> execute() async {
    await localDataSource.clearCache();
  }
}
```

```
// lib/features/cart/domain/usecases/checkout.dart

@lazySingleton
class Checkout {
  final CartRepository cartRepository;
  final ClearProductCache clearProductCache; // ✅ UseCase, no Repository

  Checkout(this.cartRepository, this.clearProductCache);

  Future<Either<Failure, CheckoutResult>> execute() async {
    return cartRepository.checkout().flatMap((order) async {
      await clearProductCache.execute(); // ✅ Acción de otra feature
      return Right(CheckoutResult(orderId: order.id));
    });
  }
}
```

¿Por qué es mejor? - **Checkout** solo sabe que existe **ClearProductCache** - No sabe si viene de Firebase, API, o Base de Datos - Si cambias la implementación de **ClearProductCache**, **Checkout** no se entera

2.3 Patrón 3: Event Bus (Pub/Sub)

Cuándo usarlo: Cuando necesitas comunicación **asíncrona** y **desacoplada** entre múltiples features.

Estructura:

```
Feature_A dispara evento
    ↓
Event Bus (Core)
    ↓
Feature_B, C, D escuchan evento
```

Implementación del Event Bus:

```
// lib/core/events/app_event.dart

abstract class AppEvent {}

class UserLoggedOutEvent extends AppEvent {}

class UserLoggedInEvent extends AppEvent {
  final User user;
  UserLoggedInEvent(this.user);
}

class ProductAddedToCartEvent extends AppEvent {
  final String productId;
  final int quantity;
  ProductAddedToCartEvent(this.productId, this.quantity);
}
```

```
// lib/core/events/event_bus.dart

class EventBus {
  final _events = StreamController<AppEvent>.broadcast();

  Stream<T> on<T extends AppEvent>() {
    return _events.stream.where((e) => e is T).cast<T>();
  }

  void fire(AppEvent event) {
    _events.add(event);
  }
}

// Registro como singleton
final eventBus = EventBus();
```

Uso: Disparar Eventos:

```
// lib/features/auth/presentation/cubit/auth_cubit.dart

@injectable
class AuthCubit extends Cubit<AuthState> {
  final AuthRepository authRepository;
  final EventBus eventBus;

  AuthCubit(this.authRepository, this.eventBus) :
    super(AuthInitial());

  Future<void> signOut() async {
    final result = await authRepository.signOut();

    result.match(
      (failure) => emit(AuthError(failure.message)),
      (_) {
        eventBus.fire(UserLoggedOutEvent()); // ✅ Notificar a otros
        emit(AuthUnauthenticated());
      },
    );
  }
}
```

Uso: Escuchar Eventos:

```
// lib/features/cart/presentation/cubit/cart_cubit.dart

@injectable
class CartCubit extends Cubit<CartState> {
  final EventBus eventBus;
  StreamSubscription? _subscription;

  CartCubit(this.eventBus) : super(CartInitial());

  void startListening() {
    _subscription?.cancel();
    _subscription = eventBus.on<UserLoggedOutEvent>().listen(_) {
      // Limpiar carrito cuando usuario cierre sesión
      emit(CartEmpty());
    };
  }

  @override
  Future<void> close() {
    _subscription?.cancel();
    return super.close();
  }
}
```

2.4 Patrón 4: State Management Compartido

Cuándo usarlo: Cuando necesitas estado global accesible desde múltiples features.

Con Cubits Globales:

```
// lib/core/cubits/theme_cubit.dart

@injectable
class ThemeCubit extends Cubit<ThemeState> {
  ThemeCubit() : super(ThemeInitial());

  void toggleTheme() {
    final newTheme = state is DarkTheme ? LightTheme() : DarkTheme();
    emit(newTheme);
  }
}
```

```
// Uso en cualquier feature
class ProductCard extends StatelessWidget {
  final ThemeCubit themeCubit; // Inyectar

  @override
  Widget build(BuildContext context) {
    return BlocBuilder(
      bloc: themeCubit,
      builder: (context, theme) {
        return Container(
          color: theme.background,
          child: Text('Producto'),
        );
      },
    );
  }
}
```

3. Arquitectura de Features: Estructura Completa

3.1 Estructura de Carpetas Recomendada

```
lib/
├── core/
│   ├── cubits/                # Cubits globales
│   │   ├── theme_cubit.dart
│   │   └── locale_cubit.dart
│   ├── events/               # Event Bus
│   │   ├── app_event.dart
│   │   └── event_bus.dart
│   ├── domain/
│   │   ├── repositories/     # Repos compartidos
│   │   │   ├── auth_repository.dart
│   │   │   └── analytics_repository.dart
│   │   └── services/         # Core services
│   │       └── analytics/
│   └── features/
│       ├── auth/
│       │   ├── data/
│       │   │   ├── datasources/
│       │   │   └── repositories/
│       │   ├── domain/
│       │   │   ├── entities/
│       │   │   ├── repositories/
│       │   │   └── usecases/
│       │   └── presentation/
│       │       ├── cubit/
│       │       ├── widgets/
│       │       └── pages/
│       ├── products/
│       │   └── ...
│       └── cart/
│           └── ...
```

3.2 Feature como Módulo

Cada feature debe ser independiente:

```
// lib/features/cart/cart.dart

class CartFeature {
  static Widget page() {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => getIt<CartCubit>()),
      ],
      child: CartPage(),
    );
  }
}
```


4. Errores Comunes y Anti-Patrones

Error 1: Presentation Layer a Presentation Layer

```
// ❌ NUNCA hagas esto
class ProductDetailCubit extends Cubit<ProductDetailState> {
  final CartCubit cartCubit; // ⚠️ Feature A conoce Feature B

  void addToCart(Product product) {
    cartCubit.add(product); // Acoplamiento total
  }
}
```

Solución: Usar Event Bus o UseCase compartido.

Error 2: Repositorio de una Feature en otra Feature

```
// ❌ NUNCA imports así
import 'features/products/data/repositories/product_repository_impl.dart';
```

Solución: Crear interfaz en `core/domain/repositories`.

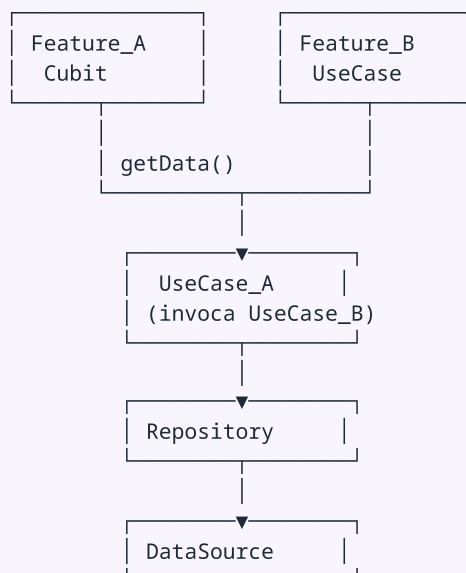
Error 3: Ciclos de Dependencia

Feature_A → Feature_B → Feature_A → ☀️

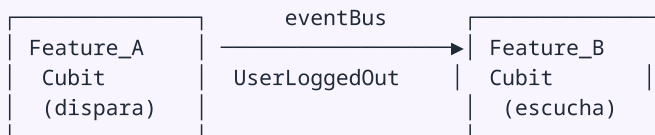
Solución: Usar Event Bus para romper el ciclo.

5. Flujo de Datos entre Features: Diagramas

5.1 Comunicación Síncrona (UseCase)



5.2 Comunicación Asíncrona (Event Bus)



6. Casos de Uso Reales

Caso 1: Carrito necesita saber si usuario está logueado

✗ **Mal:** `CartCubit` importa `AuthRepositoryImpl`

✓ **Bien:** `CartCubit` usa `AuthRepository` (interfaz en core)

```
@injectable
class CartCubit extends Cubit<CartState> {
    final AuthRepository authRepository; // ✓ Interfaz
    final AddToCart addToCart;

    CartCubit(this.authRepository, this.addToCart);

    Future<void> addProduct(Product product) async {
        final user = await authRepository.getCurrentUser();

        user.match(
            () => emit(CartNeedsLogin()), // Redirigir a login
            (_) => addToCart.execute(product),
        );
    }
}
```

Caso 2: Checkout necesita limpiar cache de productos

✓ **UseCase Delegation:**

```
class CheckoutUseCase {
    final CartRepository cartRepository;
    final ClearProductCache clearProductCache;

    // Dependency Injection de UseCase de otra feature
    CheckoutUseCase(this.cartRepository, this.clearProductCache);
}
```

Caso 3: Notificaciones necesitan saber cuando cambia el usuario

✓ **Event Bus:**

```
// En AuthCubit
eventBus.fire(UserChangedEvent(user));

// En NotificationsCubit
eventBus.on<UserChangedEvent>().listen((event) {
  subscribeToUserNotifications(event.user.id);
});
```

7. Testing de Features Desacopladas

Unit Test de UseCase con Dependencias

```
void main() {
  late Checkout checkout;
  late MockCartRepository mockCartRepo;
  late MockClearProductCache mockClearCache;

  setUp(() {
    mockCartRepo = MockCartRepository();
    mockClearCache = MockClearProductCache();

    // ✅ Inyectar mocks
    checkout = Checkout(mockCartRepo, mockClearCache);
  });

  test('checkout clears product cache', () async {
    when(() => mockCartRepo.checkout())
      .thenAnswer((_) async => Right(Order(id: '123')));
    when(() => mockClearCache.execute())
      .thenAnswer((_) async {});

    await checkout.execute();

    verify(() => mockClearCache.execute()).called(1);
  });
}
```

Integración con Event Bus

```
test('UserLoggedOutEvent clears cart', () async {
  final eventBus = EventBus();
  final cartCubit = CartCubit(eventBus);

  // Emitir evento
  eventBus.fire(UserLoggedOutEvent());

  // Verificar estado
  expect(cartCubit.state, CartEmpty());
});
```

7.1 Testing de UseCase con Dependencias Entre Features

```
// test/features/cart/domain/usecases/checkout_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/cart/domain/repositories/cart_repository.dart';
import 'package:my_app/features/cart/domain/usecases/checkout.dart';
import 'package:my_app/features/products/domain/usecases/clear_product_cache.dart';

class MockCartRepository extends Mock implements CartRepository {}
class MockClearProductCache extends Mock implements ClearProductCache {}

void main() {
  late Checkout useCase;
  late MockCartRepository mockCartRepo;
  late MockClearProductCache mockClearCache;

  setUp(() {
    mockCartRepo = MockCartRepository();
    mockClearCache = MockClearProductCache();
    useCase = Checkout(mockCartRepo, mockClearCache);
  });

  setUpAll(() {
    registerFallbackValue(const CheckoutParams());
  });

  group('Checkout UseCase', () {
    const tOrder = Order(id: 'order-123', total: 100.0);
    const tParams = CheckoutParams();

    test('debería limpiar cache después de checkout exitoso', () {
      async {
        // Arrange
        when(() => mockCartRepo.checkout(tParams))
          .thenAnswer((_) async => const Right(tOrder));
        when(() => mockClearCache.execute())
          .thenAnswer((_) async {});

        // Act
        final result = await useCase(tParams);

        // Assert
        expect(result.isRight(), true);

        // Verificar orden: checkout primero, luego clear
        verifyInOrder([
          () => mockCartRepo.checkout(tParams),
          () => mockClearCache.execute(),
        ]);
        verifyNoMoreInteractions(mockCartRepo);
        verifyNoMoreInteractions(mockClearCache);
      });

    test('debería retornar failure si checkout falla', () async {
      // Arrange
      when(() => mockCartRepo.checkout(tParams))
        .thenAnswer((_) async => const Left(CheckoutFailure('Stock
        insuficiente')));
      when(() => mockClearCache.execute())
        .thenAnswer((_) async {});

      // Act
      final result = await useCase(tParams);

      // Assert
```

```

        expect(result.isLeft(), true);
        verifyNever(() => mockClearCache.execute());
    });

    test('debería fallar si clear cache falla pero checkout exitoso',
        () async {
            // Arrange
            when(() => mockCartRepo.checkout(tParams))
                .thenAnswer((_) async => const Right(tOrder));
            when(() => mockClearCache.execute())
                .thenThrow(Exception('Cache error'));

            // Act
            final result = await useCase(tParams);

            // Assert
            expect(result.isLeft(), true);
        });
    });
}

```

7.2 Testing de Event Bus

```
// test/core/events/event_bus_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/core/events/app_event.dart';
import 'package:my_app/core/events/event_bus.dart';

void main() {
  late EventBus eventBus;

  setUp(() {
    eventBus = EventBus();
  });

  group('EventBus', () {
    test('debería recibir evento disparado', () async {
      // Arrange
      final future = eventBus.on<UserLoggedInEvent>().first;

      // Act
      eventBus.fire(UserLoggedInEvent(User(id: '1', name: 'Test')));

      // Assert
      final event = await future;
      expect(event.user.id, '1');
    });

    test('debería filtrar por tipo de evento', () async {
      // Arrange
      final subscription = eventBus.on<UserLoggedInEvent>().listen((_) {});

      // Act
      eventBus.fire(UserLoggedOutEvent());
      eventBus.fire(UserLoggedInEvent(User(id: '1', name: 'Test')));

      // Assert: El stream debería tener el evento correcto
      final event = await eventBus.on<UserLoggedInEvent>().first;
      expect(event, isA<UserLoggedInEvent>());
    });

    test('debería permitir múltiples suscriptores', () async {
      // Arrange
      int callCount = 0;
      final sub1 = eventBus.on<UserLoggedInEvent>().listen((_) => callCount++);
      final sub2 = eventBus.on<UserLoggedInEvent>().listen((_) => callCount++);

      // Act
      eventBus.fire(UserLoggedInEvent(User(id: '1', name: 'Test')));

      // Assert
      await Future.delayed(Duration.zero);
      expect(callCount, 2);

      await sub1.cancel();
      await sub2.cancel();
    });

    test('debería cerrar correctamente', () async {
      final subscription = eventBus.on<UserLoggedInEvent>().listen((_) {});

      await subscription.cancel();

      // No debería haber más eventos después de cancelar
      eventBus.fire(UserLoggedInEvent(User(id: '1', name: 'Test')));
    });
  });
}
```

```
final future = eventBus.on<UserLoggedInEvent>().first;
await Future.delayed(Duration(milliseconds: 100));

// Si no hay evento, el test pasa (subscription cancelada)
});
});
}
```

7.3 Testing de Comunicación entre Features

```
// test/integration/feature_communication_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:my_app/features/auth/domain/repositories/auth_repository.dart';
import 'package:my_app/features/auth/domain/usecases/sign_out.dart';
import 'package:my_app/features/cart/domain/usecases/clear_cart.dart';
import 'package:my_app/features/cart/presentation/cubit/cart_cubit.dart';
import 'package:my_app/core/events/event_bus.dart';

class MockAuthRepository extends Mock implements AuthRepository {}
class MockClearCart extends Mock implements ClearCart {}

void main() {
  late EventBus eventBus;
  late CartCubit cartCubit;

  setUp(() {
    eventBus = EventBus();
    cartCubit = CartCubit(eventBus);
  });

  tearDown(() {
    cartCubit.close();
  });

  group('Feature Communication Integration', () {
    test('signOut debería limpiar el carrito', () async {
      // Arrange
      // Simular que el carrito tiene productos
      // (En un caso real, el CartCubit tendría estado inicial)

      // Act
      // Disparar evento como lo haría AuthCubit
      eventBus.fire(UserLoggedOutEvent());

      // Wait para que el evento se procese
      await Future.delayed(Duration.zero);

      // Assert - El carrito debería estar vacío después de logout
      expect(cartCubit.state, CartEmpty());
    });

    test('múltiples features deberían reaccionar al mismo evento', ()
      async {
      // Arrange
      final analyticsCubit = AnalyticsCubit(eventBus);
      final notificationCubit = NotificationCubit(eventBus);

      // Act
      eventBus.fire(UserLoggedInEvent(User(id: '1', name: 'Test')));

      // Wait
      await Future.delayed(Duration.zero);

      // Assert
      expect(analyticsCubit.state, AnalyticsTrackingEnabled());
      expect(notificationCubit.state, NotificationsSubscribed());

      await analyticsCubit.close();
      await notificationCubit.close();
    });
  });
}
```


7.4 Testing de Shared Repository

```
// test/integration/shared_repository_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/domain/repositories/auth_repository.dart';
import 'package:my_app/features/perfil/domain/usecases/get_profile.dart';
import 'package:my_app/features/settings/domain/usecases/get_settings.dart';

class MockAuthRepository extends Mock implements AuthRepository {}

void main() {
  late MockAuthRepository mockAuthRepo;
  late GetProfile getProfile;
  late GetSettings getSettings;

  setUp(() {
    mockAuthRepo = MockAuthRepository();
    getProfile = GetProfile(mockAuthRepo);
    getSettings = GetSettings(mockAuthRepo);
  });

  group('Shared Repository Pattern', () {
    const tUser = User(id: '1', name: 'Test User');

    test('múltiples features deberían poder usar el mismo repositorio', () async {
      // Arrange
      when(() => mockAuthRepo.getCurrentUser())
        .thenAnswer((_) async => const Option.of(tUser));

      // Act & Assert - Feature Perfil
      final profileResult = await getProfile();
      expect(profileResult.isRight(), true);

      // Act & Assert - Feature Settings
      final settingsResult = await getSettings();
      expect(settingsResult.isRight(), true);

      // Verificar que se llamó al mismo repositorio
      verify(() => mockAuthRepo.getCurrentUser()).called(2);
    });

    test('cambios en AuthRepository deberían reflejarse en todas las features', () async {
      // Arrange
      when(() => mockAuthRepo.getCurrentUser())
        .thenAnswer((_) async => const Option.of(tUser));

      // Act
      final profile1 = await getProfile();
      final profile2 = await getProfile();

      // Assert - Mismo resultado
      profile1.match(
        (_) => fail('No debería fallar'),
        (p) => expect(p.user.name, 'Test User'),
      );
      profile2.match(
        (_) => fail('No debería fallar'),
        (p) => expect(p.user.name, 'Test User'),
      );
    });
  });
}
```

7.5 Testing de State Compartido

```
// test/core/cubits/theme_cubit_test.dart

import 'package:bloc_test/bloc_test.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/core/cubits/theme_cubit.dart';

void main() {
  late ThemeCubit themeCubit;

  setUp(() {
    themeCubit = ThemeCubit();
  });

  tearDown(() {
    themeCubit.close();
  });

  group('ThemeCubit - Shared State', () {
    test('estado inicial es tema claro', () {
      expect(themeCubit.state, LightTheme());
    });

    blocTest<ThemeCubit, ThemeState>(
      'debería cambiar a tema oscuro',
      build: () => ThemeCubit(),
      act: (cubit) => cubit.toggleTheme(),
      expect: () => [DarkTheme()],
    );

    blocTest<ThemeCubit, ThemeState>(
      'debería alternar entre temas',
      build: () => ThemeCubit(),
      act: (cubit) {
        cubit.toggleTheme();
        cubit.toggleTheme();
      },
      expect: () => [DarkTheme(), LightTheme()],
    );

    test('múltiples listeners deberían recibir el mismo estado', ()
      async {
        // Arrange
        final states = <ThemeState>[];
        themeCubit.stream.listen((state) => states.add(state));

        // Act
        themeCubit.toggleTheme();
        await Future.delayed(Duration.zero);

        // Assert
        expect(states.length, 2); // Initial + DarkTheme
      });
  });
}
```

7.6 Errores Comunes en Testing de Features

```
// ❌ Error 1: Olvidar cerrar suscripciones
tearDown() {
    // CRASH: Stream not closed
};

// ✅ Solución
tearDown() async {
    await cubit.close();
};

// ❌ Error 2: Mock no configurado
when(() => mockRepo.getUser()).thenAnswer(_) async => null);
// CRASH: MissingStubError

// ✅ Solución
setUpAll() {
    registerFallbackValue(UserParams('test'));
};

// ❌ Error 3: No aislar tests de Event Bus
test('test1', () async {
    eventBus.fire(UserLoggedInEvent()); // Afecta test2
});

// ✅ Solución - crear nuevo EventBus por test
setUp() {
    eventBus = EventBus();
};

// ❌ Error 4: Test de Feature sin mockear dependencia
test('feature test', () {
    // Usa implementación real → frágil
});

// ✅ Solución - usar mocks
setUp() {
    mockRepo = MockRepo();
    cubit = Cubit(mockRepo);
};
```

7.7 Estructura de Tests para Features

```
test/
├── features/
│   ├── auth/
│   │   ├── data/
│   │   │   └── repositories/
│   │   │       └── auth_repository_impl_test.dart
│   │   ├── domain/
│   │   │   └── usecases/
│   │   │       └── sign_out_test.dart
│   │   └── presentation/
│   │       └── cubit/
│   │           └── auth_cubit_test.dart
│   ├── cart/
│   │   └── ...
│   └── products/
│       └── ...
├── core/
│   ├── events/
│   │   └── event_bus_test.dart
│   └── cubits/
│       └── theme_cubit_test.dart
├── integration/
│   ├── feature_communication_test.dart
│   └── shared_repository_test.dart
└── mocks/
    ├── mock_classes.dart
    └── fixtures/
        ├── user_fixture.dart
        └── product_fixture.dart
```

8. Mejores Prácticas

Reglas de Oro

1. Nunca importes Presentation de otra feature
2. Usa Interfaces en `core/domain/repositories` para comunicación de datos
3. Usa UseCases compartidos para acciones de negocio
4. Usa Event Bus para eventos globales (login, logout, temas)
5. Una feature es un módulo - Puede deshabilitarse/reescribirse completamente

Checklist antes de hacer Feature Communication

- ☐ ¿Puedo lograr esto con una interfaz en core?
- ☐ ¿El uso es de datos → Repository?
- ☐ ¿El uso es de acción → UseCase?
- ☐ ¿Es un evento que múltiples features necesitan saber → Event Bus?
- ☐ ¿Es estado global → Cubit compartido?

9. Comunicación con Módulos Externos

Feature → Firebase/Backend

No uses el SDK directamente en los Cubits. Capa de abstracción:

Cubit → Repository (interfaz) → RepositoryImpl → SDK

Feature → Device (Cámara, GPS, Sensores)

Cubit → DeviceService (interfaz) → DeviceServiceImpl → Platform Channels

10. Resumen de Patrones

Patrón	Cuándo Usar	Complejidad
Shared Repository	Múltiples features leen mismos datos	Baja
UseCase Delegation	Feature ejecuta acción de otra	Baja
Event Bus	Eventos asíncronos, uno → muchos	Media
Shared State	Estado global (tema, locale)	Baja

Resumen Ejecutivo

1. El acoplamiento destruye Clean Architecture - Evita imports directos entre features
2. Usa Interfaces en `core/domain/repositories` para compartir datos
3. Usa UseCases para compartir acciones de negocio
4. Usa Event Bus para comunicación asíncrona y eventos globales
5. **NUNCA** ties Presentation a Presentation
6. **Testing** se simplifica dramáticamente con dependencias inyectables

Siguiente nivel: Aprende a diseñar features completamente independientes que se comunican solo a través de eventos, el patrón "Micro-frontends" aplicado a Flutter.

 **Ejemplo de orquestación Multi-Bloc desde la UI:** [16-BLOC-CUBIT/10-ejemplo-multi-bloc.md](#) —
DashboardCubit coordinando 3 cubits con MultiBlocProvider y MultiBlocListener. **Concurrencia de eventos:** [16-BLOC-CUBIT/13-concurrencia-eventos.md](#) — droppable, restartable, debounce, sequential.

Recursos Adicionales

- The Power of Event Bus
 - Modular Architecture in Flutter
 - BLoC Communication
-

Ver también

- [16-BLOC-CUBIT](#) — MultiBlocProvider y MultiBlocListener
 - [13-MANUAL-JSON](#) — Serialización manual para modelos complejos
-

En el siguiente módulo

→ [04-streams-tiempo-real.md](#) — Patrones reactivos con Streams y StreamUseCase

Nivel Experto

Nivel Experto: Streams y Tiempo Real con Clean Architecture

Las aplicaciones modernas no son estáticas. Un chat actualiza en tiempo real, un dashboard financiero muestra cotizaciones Live, una app de delivery tracks el repartidor. En Clean Architecture tradicional, todo es `Future<Either<Failure, T>>`. Pero cuando los datos fluyen continuamente, necesitamos `StreamUseCase`: el equivalente reactivo de los UseCases tradicionales.

1. Fundamentos: Por Qué Streams en Clean Architecture

1.1 El Problema con Future

`Future` representa un valor único en el futuro. Pero ¿qué pasa cuando hay múltiples valores?

```
// ❌ Future no sirve para datos continuos
Future<List<Message>> getMessages() async {
  return await api.getMessages(); // Solo obtiene UNA vez
}

// Si llegan mensajes nuevos, ¿cómo los recibe el UI?
```

1.2 Streams al Rescate

```
// ✅ Stream representa un flujo de valores
Stream<List<Message>> watchMessages() {
  return api.watchMessages(); // Notificaciones continuas
}
```

1.3 Cuándo Usar Streams

Escenario	Tipo de Datos	Herramienta
API REST tradicional	Una respuesta	<code>Future<Either<Failure, T>></code>
Chat/Mensajería	Múltiples mensajes	<code>Stream<Either<Failure, T>></code>
Precios/Stock	Actualizaciones frecuentes	<code>Stream<T></code>
Auth State	Sesión activa	<code>Stream<User?></code>
Notificaciones	Push events	<code>Stream<Notification></code>
Ubicación	GPS coordinates	<code>Stream<Location></code>

2. StreamUseCase: El Contrato Base

2.1 Definición

```
// lib/core/common/stream_usecase.dart

import 'package:fpdart/fpdart.dart';
import 'package:my_app/core/error/failures.dart';

/// Contrato base para UseCases que retornan Streams
///
/// Útil para:
/// - Firebase/Firestore watchers
/// - WebSocket connections
/// - SSE (Server-Sent Events)
abstract class StreamUseCase<Type, Params> {
  /// Ejecuta el stream con los parámetros dados
  Stream<Either<Failure, Type>> call(Params params);
}
```

2.2 Interfaz Genérica sin Params

```
// Para streams que no necesitan parámetros
abstract class StreamUseCaseNoParams<Type> {
  Stream<Either<Failure, Type>> call();
}
```


3. Implementación Completa: Chat en Tiempo Real

3.1 Domain: Entities y Repositories

```
// lib/features/chat/domain/entities/message.dart
```

```
class Message {  
  final String id;  
  final String chatId;  
  final String senderId;  
  final String content;  
  final DateTime timestamp;  
  final MessageStatus status;  
  
  const Message({  
    required this.id,  
    required this.chatId,  
    required this.senderId,  
    required this.content,  
    required this.timestamp,  
    required this.status,  
  });  
}  
  
enum MessageStatus { sending, sent, delivered, read }
```

```
// lib/features/chat/domain/repositories/chat_repository.dart
```

```
abstract class ChatRepository {  
  /// Stream de mensajes de un chat específico  
  Stream<Either<Failure, List<Message>>> watchMessages(String chatId);  
  
  /// Stream de múltiples chats  
  Stream<Either<Failure, List<ChatSummary>>> watchChats();  
  
  /// Enviar mensaje (Future tradicional)  
  Future<Either<Failure, Message>> sendMessage(SendMessageParams params);  
  
  /// Marcar como leído  
  Future<Either<Failure, void>> markAsRead(String messageId);  
}
```

3.2 Domain: UseCases

```
// lib/features/chat/domain/usecases/watch_messages.dart
```

```
@lazySingleton  
class WatchMessages extends StreamUseCase<List<Message>, String> {  
  final ChatRepository repository;  
  
  WatchMessages(this.repository);  
  
  @override  
  Stream<Either<Failure, List<Message>>> call(String chatId) {  
    return repository.watchMessages(chatId);  
  }  
}
```

```
// lib/features/chat/domain/usecases/watch_chats.dart

@lazySingleton
class WatchChats extends StreamUseCaseNoParams<List<ChatSummary>> {
  final ChatRepository repository;

  WatchChats(this.repository);

  @override
  Stream<Either<Failure, List<ChatSummary>>> call() {
    return repository.watchChats();
  }
}
```

```
// lib/features/chat/domain/usecases/filter_active_messages.dart

/// Ejemplo de lógica de negocio en el Stream
@lazySingleton
class FilterActiveMessages
  extends StreamUseCase<List<Message>, String> {
  final ChatRepository repository;

  FilterActiveMessages(this.repository);

  @override
  Stream<Either<Failure, List<Message>>> call(String chatId) {
    // Transformar el stream: filtrar mensajes no eliminados
    return repository.watchMessages(chatId).map((result) {
      return result.map((messages)
        => messages.where((m) => !m.isDeleted).toList()
      );
    });
  }
}
```

3.3 Data: Repository Implementation

```
// lib/features/chat/data/repositories/chat_repository_impl.dart

@LazySingleton(as: ChatRepository)
class ChatRepositoryImpl implements ChatRepository {
  final MessageRemoteDataSource remoteDataSource;

  ChatRepositoryImpl(this.remoteDataSource);

  @override
  Stream<Either<Failure, List<Message>>> watchMessages(String chatId)
  {
    // Convertir Stream<DataModel> a Stream<Either<Failure, Entity>>
    return remoteDataSource.watchMessages(chatId).map((models) {
      try {
        final messages = models.map((m) => m.toEntity()).toList();
        return Right<Failure, List<Message>>(messages);
      } catch (e) {
        return Left<Failure, List<Message>>(
          ParseFailure('Error parseando mensajes: $e'),
        );
      }
    }).handleError((error) {
      // Manejar errores del Stream (Firebase desconectado, etc.)
      if (error is SocketException) {
        return Left<Failure, List<Message>>(NetworkFailure('Sin conexión'));
      }
      return Left<Failure, List<Message>>(ServerFailure(error.toString()));
    });
  }

  @override
  Stream<Either<Failure, List<ChatSummary>>> watchChats() {
    return remoteDataSource.watchChats().map((models) {
      return Right<Failure, List<ChatSummary>>(models.map((m) => m.toEntity()).toList());
    }).handleError((e) => Left<Failure, List<ChatSummary>>(ServerFailure(e.toString())));
  }

  @override
  Future<Either<Failure, Message>> sendMessage(SendMessageParams params) async {
    try {
      final model = await remoteDataSource.sendMessage(params);
      return Right<Failure, Message>(model.toEntity());
    } catch (e) {
      return Left<Failure, Message>(ServerFailure(e.toString()));
    }
  }
}
```

3.4 Data: Firebase/Firestore Implementation

```
// lib/features/chat/data/datasources/message_remote_datasource.dart

abstract class MessageRemoteDataSource {
  Stream<List<MessageModel>> watchMessages(String chatId);
  Stream<List<ChatSummaryModel>> watchChats();
  Future<MessageModel> sendMessage(SendMessageParams params);
}
```

```
// lib/features/chat/data/datasources/message_remote_datasource_impl.dart

@lazySingleton
class MessageRemoteDataSourceImpl implements MessageRemoteDataSource
{
  final FirebaseFirestore _firestore;

  MessageRemoteDataSourceImpl(this._firestore);

  @override
  Stream<List<MessageModel>> watchMessages(String chatId) {
    // Firestore ya retorna un Stream
    return _firestore
      .collection('chats')
      .doc(chatId)
      .collection('messages')
      .orderBy('timestamp', descending: true)
      .snapshots() // ← Este es el Stream de Firebase
      .map((snapshot) => snapshot.docs
        .map((doc) => MessageModel.fromJson(doc.data()))
        .toList());
  }

  @override
  Stream<List<ChatSummaryModel>> watchChats() {
    return _firestore
      .collection('chats')
      .where('participants', arrayContains: _firestore.uid)
      .snapshots()
      .map((snapshot) => snapshot.docs
        .map((doc) => ChatSummaryModel.fromJson(doc.data()))
        .toList());
  }
}

```

4. Presentation: Consumiendo Streams

4.1 Cubit con StreamSubscription

```
// lib/features/chat/presentation/cubit/chat_cubit.dart

@injectable
class ChatCubit extends Cubit<ChatState> {
  final WatchMessages watchMessages;
  final SendMessage sendMessage;
  final ChatRepository chatRepository;

  StreamSubscription? _subscription;
  String? _currentChatId;

  ChatCubit(
    this.watchMessages,
    this.sendMessage,
    this.chatRepository,
  ) : super(ChatInitial());

  void watchChat(String chatId) {
    _currentChatId = chatId;
    _subscription?.cancel(); // Limpiar stream anterior

    _subscription = watchMessages(chatId).listen((result) {
      result.match(
        (failure) => emit(ChatError(failure.message)),
        (messages) => emit(ChatLoaded(messages)),
      );
    });
  }

  void sendMessage(String content) async {
    if (_currentChatId == null) return;

    emit(ChatSending());

    final result = await sendMessage(SendMessageParams(
      chatId: _currentChatId!,
      content: content,
    ));

    result.match(
      (failure) => emit(ChatError(failure.message)),
      (_) => emit(ChatMessageSent()),
    );
  }

  @override
  Future<void> close() {
    _subscription?.cancel(); // ⚠️ CRÍTICO: Prevenir memory leaks
    return super.close();
  }
}
```

4.2 Errores: Memory Leaks

El error más común: Olvidar cancelar la suscripción.

```
// ❌ Memory Leak - No cancelar
class BadCubit extends Cubit<State> {
    StreamSubscription? _sub;

    void start() {
        _sub = stream.listen((v) => emit(v));
        // Nunca se cancela ☀️
    }

    // No overrides close
}

// ✅ Correcto - Siempre cancelar
class GoodCubit extends Cubit<State> {
    StreamSubscription? _sub;

    void start() {
        _sub = stream.listen((v) => emit(v));
    }

    @override
    Future<void> close() {
        _sub?.cancel(); // ✅ Se limpia
        return super.close();
    }
}
```

5. Operadores de Streams: Transformaciones Avanzadas

5.1 Filtrado (where)

```
// Solo mensajes de un usuario específico
@lazySingleton
class WatchUserMessages extends StreamUseCase<List<Message>,
    UserMessagesParams> {
    final ChatRepository repository;

    WatchUserMessages(this.repository);

    @override
    Stream<Either<Failure, List<Message>>> call(UserMessagesParams
        params) {
        return repository.watchMessages(params.chatId).map((result) {
            return result.map((messages)
                => messages.where((m) => m.senderId ==
                    params.userId).toList()
            );
        });
    }
}
```

5.2 Mapeo (map)

```
// Transformar a DTO para UI
@lazySingleton
class MapMessagesToDisplay
    extends StreamUseCase<List<MessageDisplay>, String> {
    final WatchMessages watchMessages;

    MapMessagesToDisplay(this.watchMessages);

    @override
    Stream<Either<Failure, List<MessageDisplay>>> call(String chatId) {
        return watchMessages(chatId).map((result) {
            return result.map((messages) => messages.map((m) =>
                MessageDisplay(
                    id: m.id,
                    text: m.content,
                    time: _formatTime(m.timestamp),
                    isMe: m.senderId == currentUserId,
                    status: _mapStatus(m.status),
                )).toList());
        });
    }

    String _formatTime(DateTime dt) => ...;
    MessageDisplayStatus _mapStatus(MessageStatus s) => ...;
}
```

5.3 Debounce (esperar input quieto)

```
// Búsqueda en tiempo real - no buscar en cada keystroke
class SearchProductsStream extends StreamUseCase<List<Product>,
    String> {
    final ProductRepository repository;

    SearchProductsStream(this.repository);

    @override
    Stream<Either<Failure, List<Product>>> call(String query) {
        // Crear stream desde TextField
        return Stream.periodic(const Duration(milliseconds: 300))
            .map((_) => query) // Obtener último valor
            .distinct() // Solo si cambió
            .debounceTime(const Duration(milliseconds: 500)) // ✓
            // Esperar 500ms sin input
            .asyncMap((q) => repository.searchProducts(q))
            .handleError((e) => Left(ServerFailure(e.toString())));
    }
}
```

5.4 Distinct (evitar duplicados)

```
// No emitir el mismo valor dos veces
stream.distinct().listen((value) {
    print('Nuevo valor: $value');
});
```

5.5 Take y Skip

```
// Pagination con streams
@lazySingleton
class PaginateMessages extends StreamUseCase<List<Message>,
    PaginationParams> {
    final WatchMessages watchMessages;

    PaginateMessages(this.watchMessages);

    @override
    Stream<Either<Failure, List<Message>>> call(PaginationParams
        params) {
        return watchMessages(params.chatId).map((result) {
            return result.map((messages) {
                final start = params.page * params.pageSize;
                final end = start + params.pageSize;
                return messages.skip(start).take(params.pageSize).toList();
            });
        });
    }
}
```

6. Firebase/Firestore: Casos de Uso Reales

6.1 Reconexión Automática

```
class FirestoreMessageDataSource implements MessageRemoteDataSource {
    final FirebaseFirestore _firestore;
    StreamSubscription? _connectionSubscription;
    bool _isConnected = true;

    FirestoreMessageDataSource(this._firestore) {
        // Monitorear conexión
        FirebaseDatabase.instance.ref('.info/connected').onValue.listen((event) {
            _isConnected = event.snapshot.value == true;
        });
    }

    @override
    Stream<List<MessageModel>> watchMessages(String chatId) {
        final controller = StreamController<List<MessageModel>>();

        final sub = _firestore
            .collection('chats')
            .doc(chatId)
            .collection('messages')
            .snapshots()
            .listen(
                (snapshot) {
                    if (_isConnected) {
                        controller.add(snapshot.docs
                            .map((d) => MessageModel.fromJson(d.data()))
                            .toList());
                    } else {
                        controller.addError('Sin conexión');
                    }
                },
                onError: controller.addError,
            );

        controller.onCancel = () => sub.cancel();

        return controller.stream;
    }
}
```

6.2 Optimización: Escuchar Solo Cambios

```
// En lugar de obtener todo, escuchar cambios incrementales
_stream = _firestore
    .collection('messages')
    .doc(messageId)
    .snapshots() // Solo este documento
    .map((doc) => doc.exists ? MessageModel.fromJson(doc.data()) :
        null);
```

6.3 Batch y Transacciones

```
Future<Either<Failure, void>> sendMessageWithTypingIndicator(
    SendMessageParams params,
) async {
    try {
        final batch = _firestore.batch();

        // 1. Agregar mensaje
        final msgRef = _firestore
            .collection('chats')
            .doc(params.chatId)
            .collection('messages')
            .doc();
        batch.set(msgRef, MessageModel(
            id: msgRef.id,
            content: params.content,
            timestamp: DateTime.now(),
        ).toJson());

        // 2. Actualizar "último mensaje" en chat
        final chatRef =
            _firestore.collection('chats').doc(params.chatId);
        batch.update(chatRef, {
            'lastMessage': params.content,
            'lastMessageAt': DateTime.now(),
        });

        await batch.commit();
        return const Right(null);
    } catch (e) {
        return Left(ServerFailure(e.toString()));
    }
}
```

7. WebSockets: Comunicación Real

7.1 DataSource con WebSocket

```
// lib/features/trading/data/datasources/price_datasource.dart

@lazySingleton
class PriceRemoteDataSource implements PriceRemoteDataSource {
  WebSocket? _socket;
  final _pricesController =
    StreamController<PriceUpdate>.broadcast();

  void connect() {
    _socket = WebSocket('wss://api.trading.com/prices');

    _socket!.listen(
      (data) {
        final update = PriceUpdate.fromJson(jsonDecode(data));
        _pricesController.add(update);
      },
      onError: (error) {
        _pricesController.addError(error);
      },
      onDone: () {
        // Reconectar después de delay
        Future.delayed(const Duration(seconds: 5), connect);
      },
    );
  }

  @override
  Stream<PriceUpdate> watchPrices(String symbol) {
    return _pricesController.stream.where((p) => p.symbol == symbol);
  }

  void disconnect() {
    _socket?.close();
  }
}
```

7.2 Repository con Reconexión

```
// lib/features/trading/data/repositories/price_repository_impl.dart

@LazySingleton(as: PriceRepository)
class PriceRepositoryImpl implements PriceRepository {
  final PriceRemoteDataSource remoteDataSource;
  final ConnectionMonitor connectionMonitor;

  PriceRepositoryImpl(this.remoteDataSource, this.connectionMonitor);

  @override
  Stream<Either<Failure, PriceUpdate>> watchPrice(String symbol) {
    // Asegurar conexión
    if (!connectionMonitor.isConnected) {
      remoteDataSource.connect();
    }

    return remoteDataSource.watchPrices(symbol).map((update) {
      return Right<Failure, PriceUpdate>(update);
    }).handleError((e) {
      return Left<Failure, PriceUpdate>(
        ConnectionFailure('Error de conexión: $e'),
      );
    });
  }
}
```

8. Backpressure: Cuando los Datos Llegan Muy Rápido

8.1 El Problema

Si recibes 1000 precios por segundo, tu UI no puede renderizar todos. Necesitas **control de flujo**.

8.2 Soluciones

```
// Opción 1: Sample - tomar cada N valores
stream.sample(const Duration(seconds: 1)).listen((value) {
  // Actualizar UI máximo 1 vez por segundo
});

// Opción 2: Throttle - limitar frecuencia
stream.throttleTime(
  const Duration(milliseconds: 500),
  trailing: true,
).listen((value) => updateUI(value));

// Opción 3: Buffer - acumular y procesar en batches
stream.bufferTime(const Duration(seconds: 1)).listen((batch) {
  // batch = lista de valores acumulados
  updateChart(batch);
});
```

9. Testing de Streams

9.1 Unit Test de StreamUseCase

```
void main() {
    late WatchMessages watchMessages;
    late MockChatRepository mockRepository;
    late StreamController<List<Message>> controller;

    setUp(() {
        mockRepository = MockChatRepository();
        controller = StreamController<List<Message>>.broadcast();
        watchMessages = WatchMessages(mockRepository);
    });

    test('watchMessages retorna stream del repository', () async {
        final messages = [
            Message(id: '1', content: 'Hello'),
        ];

        when(() => mockRepository.watchMessages('chat1'))
            .thenAnswer((_) => Stream.value(Right(messages)));

        final stream = watchMessages('chat1');

        await expectLater(
            stream,
            emits(Right(messages)),
        );
    });
}
```

9.2 Test de Cubit con Streams

```
void main() {
    late ChatCubit chatCubit;
    late StreamController<Either<Failure, List<Message>>> controller;

    setUp(() {
        controller = StreamController.broadcast();
        chatCubit = ChatCubit(watchMessages, sendMessage, repository);
    });

    test('emite ChatLoaded cuando llega mensaje', () async {
        final messages = [Message(id: '1', content: 'Test')];

        // Simular stream
        when(() => watchMessages('chat1')).thenAnswer((_) =>
            Stream.value(Right(messages))
        );

        chatCubit.watchChat('chat1');

        await Future.delayed(Duration.zero);

        expect(chatCubit.state, ChatLoaded(messages));
    });
}
```

9.3 Testing de StreamUseCase con Fake Streams

```
// test/features/chat/domain/usecases/watch_messages_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/chat/domain/repositories/chat_repository.dart';
import 'package:my_app/features/chat/domain/usecases/watch_messages.dart';

class MockChatRepository extends Mock implements ChatRepository {}

void main() {
  late WatchMessages useCase;
  late MockChatRepository mockRepository;

  setUp(() {
    mockRepository = MockChatRepository();
    useCase = WatchMessages(mockRepository);
  });

  setUpAll(() {
    registerFallbackValue('chat1');
  });

  group('WatchMessages StreamUseCase', () {
    final tMessages = [
      Message(id: '1', content: 'Hello', chatId: 'chat1', senderId:
        'user1', timestamp: DateTime.now(), status:
        MessageStatus.sent),
      Message(id: '2', content: 'World', chatId: 'chat1', senderId:
        'user2', timestamp: DateTime.now(), status:
        MessageStatus.sent),
    ];

    test('debería retornar stream del repository', () async {
      // Arrange
      when(() => mockRepository.watchMessages('chat1'))
        .thenAnswer((_) => Stream.value(Right(tMessages)));

      // Act
      final stream = useCase('chat1');

      // Assert
      await expectLater(
        stream,
        emits(Right(tMessages)),
      );
    });

    test('debería retornar error cuando repository falla', () async {
      // Arrange
      when(() => mockRepository.watchMessages('chat1'))
        .thenAnswer((_) => Stream.value(const
          Left(NetworkFailure('Sin conexión'))));

      // Act
      final stream = useCase('chat1');

      // Assert
      await expectLater(
        stream,
        emits(isA<Left<Failure, List<Message>>>()),
      );
    });

    test('debería emitir múltiples valores en secuencia', () async {
      // Arrange
```

```

final controller = StreamController<Either<Failure,
    List<Message>>>();

when(() => mockRepository.watchMessages('chat1'))
    .thenAnswer((_) => controller.stream);

final stream = useCase('chat1');

// Act & Assert
await expectLater(
    stream,
    emitsInOrder([
        Right([tMessages[0]]),
        Right(tMessages),
        Right([]),
    ]),
);

controller.add(Right([tMessages[0]]));
controller.add(Right(tMessages));
controller.add(Right([]));
await controller.close();
});
});
}

```

9.4 Testing de Repository con Streams

```
// test/features/chat/data/repositories/chat_repository_impl_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/chat/data/datasources/message_remote_datasource.dart';
import 'package:my_app/features/chat/data/repositories/chat_repository_impl.dart';
import 'package:my_app/core/error/failures.dart';

class MockMessageRemoteDataSource extends Mock implements
    MessageRemoteDataSource {}

void main() {
  late ChatRepositoryImpl repository;
  late MockMessageRemoteDataSource mockDataSource;

  setUp(() {
    mockDataSource = MockMessageRemoteDataSource();
    repository = ChatRepositoryImpl(mockDataSource);
  });

  setUpAll(() {
    registerFallbackValue('chat1');
  });

  group('watchMessages', () {
    final tModels = [
      MessageModel(id: '1', content: 'Hello', chatId: 'chat1',
        senderId: 'user1', timestamp: DateTime.now()),
    ];

    test('debería retornar stream de Right cuando datasource exitos',
      () async {
        // Arrange
        when(() => mockDataSource.watchMessages('chat1'))
          .thenAnswer((_) => Stream.value(tModels));

        // Act
        final stream = repository.watchMessages('chat1');

        // Assert
        await expectLater(
          stream,
          emits(predicate<Either<Failure, List<Message>>>((result) {
            return result.isRight();
          })),
        );
      });

    test('debería retornar Left cuando datasource lanza error', ()
      async {
        // Arrange
        when(() => mockDataSource.watchMessages('chat1'))
          .thenAnswer((_) => Stream.error(SocketException('Sin
            conexión')));

        // Act
        final stream = repository.watchMessages('chat1');

        // Assert
        await expectLater(
          stream,
          emits(predicate<Either<Failure, List<Message>>>((result) {
            return result.isLeft() &&
              result.match((f) => f is NetworkFailure, (_) =>
                false);
          })),
        );
      });
  });
}
```



```

    );
  });

test('debería mapear modelos a entidades', () async {
  // Arrange
  when(() => mockDataSource.watchMessages('chat1'))
    .thenAnswer((_) => Stream.value(tModels));

  // Act
  final stream = repository.watchMessages('chat1');

  // Assert
  await expectLater(
    stream,
    emits(predicate<Either<Failure, List<Message>>>>((result) {
      return result.match(
        (_) => false,
        (messages) => messages.first.id == '1',
      );
    })),
  );
});
}

```

9.5 Testing de Cubit con StreamSubscription

```
// test/features/chat/presentation/cubit/chat_cubit_test.dart

import 'package:bloc_test/bloc_test.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/chat/domain/usecases/watch_messages.dart';
import 'package:my_app/features/chat/domain/usecases/send_message.dart';
import 'package:my_app/features/chat/domain/repositories/chat_repository.dart';
import 'package:my_app/features/chat/presentation/cubit/chat_cubit.dart';

class MockWatchMessages extends Mock implements WatchMessages {}
class MockSendMessage extends Mock implements SendMessage {}
class MockChatRepository extends Mock implements ChatRepository {}

void main() {
  late ChatCubit cubit;
  late MockWatchMessages mockWatchMessages;
  late MockSendMessage mockSendMessage;

  setUp(() {
    mockWatchMessages = MockWatchMessages();
    mockSendMessage = MockSendMessage();
    cubit = ChatCubit(mockWatchMessages, mockSendMessage);
  });

  tearDown(() {
    cubit.close();
  });

  setUpAll(() {
    registerFallbackValue('chat1');
    registerFallbackValue(const SendMessageParams(chatId: 'chat1',
      content: 'Test'));
  });

  group('ChatCubit', () {
    final tMessages = [
      Message(id: '1', content: 'Hello', chatId: 'chat1', senderId:
        'user1', timestamp: DateTime.now(), status:
        MessageStatus.sent),
    ];

    test('estado inicial es ChatInitial', () {
      expect(cubit.state, ChatInitial());
    });

    blocTest<ChatCubit, ChatState>(
      'debería emitir ChatLoaded cuando watchChat succeeds',
      setUp: () {
        when(() => mockWatchMessages('chat1'))
          .thenAnswer((_) => Stream.value(Right(tMessages)));
      },
      build: () => ChatCubit(mockWatchMessages, mockSendMessage),
      act: (cubit) => cubit.watchChat('chat1'),
      wait: const Duration(milliseconds: 100),
      expect: () => [
        ChatLoaded(tMessages),
      ],
      verify: (_) {
        verify(() => mockWatchMessages('chat1')).called(1);
      },
    );

    blocTest<ChatCubit, ChatState>(
      'debería emitir ChatError cuando watchChat fails',
      setUp: () {

```

```

        when(() => mockWatchMessages('chat1'))
            .thenAnswer((_) => Stream.value(const
                Left(NetworkFailure('Error'))));
    },
    build: () => ChatCubit(mockWatchMessages, mockSendMessage),
    act: (cubit) => cubit.watchChat('chat1'),
    wait: const Duration(milliseconds: 100),
    expect: () => [
        const ChatError('Error'),
    ],
);

blocTest<ChatCubit, ChatState>(
    'debería cancelar suscripción anterior al cambiar de chat',
    setUp: () {
        when(() => mockWatchMessages(any()))
            .thenAnswer((_) => Stream.value(Right(tMessages)));
    },
    build: () => ChatCubit(mockWatchMessages, mockSendMessage),
    act: (cubit) async {
        cubit.watchChat('chat1');
        await Future.delayed(const Duration(milliseconds: 50));
        cubit.watchChat('chat2'); // Cambiar chat
    },
    wait: const Duration(milliseconds: 150),
    verify: (_) {
        // Solo debería llamarse una vez con el último chat
        verify(() => mockWatchMessages('chat2')).called(1);
    },
);

test('debería cancelar suscripción al cerrar', () async {
    // Arrange
    final subscription = MockStreamSubscription<Either<Failure,
        List<Message>>>();

    when(() => mockWatchMessages('chat1'))
        .thenAnswer((_) => Stream.value(Right(tMessages)));
    when(() => subscription.cancel()).thenAnswer((_) async {});

    // Act
    await cubit.watchChat('chat1');
    await cubit.close();

    // Assert - verificar que se limpió
    verifyNever(() => subscription.cancel()); // La suscripción se
        maneja internamente
    });
});
}

```

9.6 Testing con Firebase/Firestore Mock

```
// test/mocks/firebase_mocks.dart

import 'package:firebase_core/firebase_core.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

// Mock de Firebase
class MockFirebaseApp extends Mock implements FirebaseApp {}

// Mock de FirebaseAuth
class MockFirebaseAuth extends Mock implements FirebaseAuth {}

// Mock de Firestore
class MockFirestore extends Mock implements Firestore {}

// Mock de QuerySnapshot
class MockQuerySnapshot<T> extends Mock implements QuerySnapshot<T> {}

// Mock de DocumentSnapshot
class MockDocumentSnapshot<T> extends Mock implements DocumentSnapshot<T> {}

// Setup para tests
Future<void> setupFirebaseMocks() async {
  TestWidgetsFlutterBinding.ensureInitialized();

  final mockApp = MockFirebaseApp();
  when(() => mockApp.name).thenReturn('test-app');

  Firebase.initializeApp(name: 'test-app');
  when(() => Firebase.app()).thenReturn(mockApp);
}
```

9.7 Testing de Backpressure

```
// test/features/trading/domain/usecases/watch_prices_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/trading/domain/repositories/price_repository.dart';
import 'package:my_app/features/trading/domain/usecases/watch_prices.dart';

class MockPriceRepository extends Mock implements PriceRepository {}

void main() {
  late WatchPrices useCase;
  late MockPriceRepository mockRepository;

  setUp(() {
    mockRepository = MockPriceRepository();
    useCase = WatchPrices(mockRepository);
  });

  setUpAll(() {
    registerFallbackValue('BTCUSD');
  });

  group('WatchPrices Backpressure', () {
    test('debería manejar alto volumen de datos', () async {
      // Arrange - Simular 100 precios por segundo
      final controller = StreamController<Either<Failure, PriceUpdate>>();

      when(() => mockRepository.watchPrice('BTCUSD'))
        .thenAnswer((_) => controller.stream);

      final stream = useCase('BTCUSD');

      // Act - Emitir 1000 precios rápidamente
      for (int i = 0; i < 1000; i++) {
        controller.add(Right(PriceUpdate(
          symbol: 'BTCUSD',
          price: 50000 + i.toDouble(),
          timestamp: DateTime.now(),
        )));
      }

      await controller.close();

      // Assert - Verificar que el stream puede manejar la carga
      final results = await stream.take(1000).toList();
      expect(results.length, 1000);
    });

    test('debería aplicar throttle correctamente', () async {
      // Arrange - Simular stream muy rápido
      final fastController = StreamController<Either<Failure, int>>();

      when(() => mockRepository.watchPrice('BTCUSD'))
        .thenAnswer((_) => fastController.stream);

      // El throttle se aplica en el UseCase o en el Cubit
      // Aquí verificamos que el stream procese correctamente
      final results = <int>[];

      useCase('BTCUSD').listen((result) {
        result.match((_) {}, (v) => results.add(v));
      });

      // Act - Emitir 10 valores en 100ms
    });
  });
}
```

```
for (int i = 0; i < 10; i++) {
    fastController.add(Right(i));
    await Future.delayed(const Duration(milliseconds: 10));
}

await fastController.close();
await Future.delayed(const Duration(seconds: 1));

// Assert - Todos los valores fueron procesados
expect(results.length, 10);
});
});
}
```

9.8 Testing de WebSocket Reconnection

```
// test/features/trading/data/datasources/price_datasource_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:my_app/features/trading/data/datasources/price_datasource.dart';

class MockWebSocket extends Mock implements WebSocket {}

void main() {
  late PriceRemoteDataSource dataSource;

  setUp(() {
    dataSource = PriceRemoteDataSource();
  });

  group('PriceRemoteDataSource WebSocket', () {
    test('debería reconectar cuando se pierde conexión', () async {
      // Arrange
      int connectCallCount = 0;

      // Este test verificaría la lógica de reconexión
      // Simulando que el socket se cierra y se reconecta

      dataSource.connect();
      await Future.delayed(const Duration(milliseconds: 100));

      expect(connectCallCount, 1); // Conexión inicial
    });

    test('debería cerrar conexión correctamente', () async {
      dataSource.connect();
      await Future.delayed(const Duration(milliseconds: 100));

      // No debería haber errores al cerrar
      expect(() => dataSource.disconnect(), returnsNormally);
    });

    test('debería filtrar precios por símbolo', () async {
      // Arrange
      final controller = StreamController<String>.broadcast();

      // Act
      final filteredStream = controller.stream
        .where((data) => data.startsWith('BTC'))
        .map((data) => PriceUpdate.fromJson({'symbol': 'BTC',
          'price': 50000}));

      // Assert
      controller.add('ETH-5000'); // No debería aparecer
      controller.add('BTC-50000'); // Sí debería aparecer

      await expectLater(
        filteredStream.first,
        completion(predicate<PriceUpdate>((p) => p.symbol == 'BTC')),
      );

      await controller.close();
    });
  });
}
```

9.9 Errores Comunes en Testing de Streams

```
// ❌ Error 1: No cerrar StreamController
setUp(() {
  controller = StreamController(); // Nunca se cierra
});

// ✅ Solución
tearDown(() async {
  await controller?.close();
});

// ❌ Error 2: No esperar a que el stream termine
test('stream test', () async {
  final stream = useCase('param');
  // No await, el test termina antes
});

// ✅ Solución
test('stream test', () async {
  final stream = useCase('param');
  await expectLater(stream, emits(...));
});

// ❌ Error 3: Mock de stream que no es Stream
when(() => mockRepo.watchMessages('chat1'))
  .thenAnswer((_) => [Message()]); // ❌ No es Stream!

// ✅ Solución
when(() => mockRepo.watchMessages('chat1'))
  .thenAnswer((_) => Stream.value([Message()]));

// ❌ Error 4: Olvidar registerFallbackValue
when(() => mockUseCase(any())) .thenAnswer(...);
// CRASH: MissingFallbackValueError

// ✅ Solución
setUpAll(() {
  registerFallbackValue('chat1');
});
```

9.10 Herramientas de Testing para Streams

Herramienta	Uso
<code>flutter_test</code>	Testing básico
<code>bloc_test</code>	Testing de BLoC/Cubit
<code>mocktail</code>	Mocking sin generación
<code>stream_matchers</code>	Matchers para streams

9.11 Ejemplo Completo: Integration Test de Chat

```
// test/features/chat/integration/chat_integration_test.dart

import 'package:flutter_test/flutter_test.dart';
import 'package:bloc_test/bloc_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:fpdart/fpdart.dart';
import 'package:my_app/features/chat/domain/usecases/watch_messages.dart';
import 'package:my_app/features/chat/domain/usecases/send_message.dart';
import 'package:my_app/features/chat/presentation/cubit/chat_cubit.dart';

class MockWatchMessages extends Mock implements WatchMessages {}
class MockSendMessage extends Mock implements SendMessage {}

void main() {
  late ChatCubit cubit;
  late MockWatchMessages mockWatchMessages;
  late MockSendMessage mockSendMessage;

  setUp(() async {
    mockWatchMessages = MockWatchMessages();
    mockSendMessage = MockSendMessage();
    cubit = ChatCubit(mockWatchMessages, mockSendMessage);
  });

  tearDown(() async {
    await cubit.close();
  });

  setUpAll(() {
    registerFallbackValue('chat1');
    registerFallbackValue(const SendMessageParams(chatId: 'chat1',
      content: 'Test'));
  });

  group('Chat Integration', () {
    final tMessages = [
      Message(id: '1', content: 'Hello', chatId: 'chat1', senderId:
        'user1', timestamp: DateTime.now(), status:
        MessageStatus.sent),
    ];

    test('flujo completo: iniciar chat, recibir mensajes, enviar
    mensaje', () async {
      // Arrange
      when(() => mockWatchMessages('chat1'))
        .thenAnswer((_) => Stream.value(Right(tMessages)));
      when(() => mockSendMessage(any()))
        .thenAnswer((_) async => Right(tMessages.first));

      // Act & Assert - Iniciar chat
      await Future.delayed(const Duration(milliseconds: 100));
      expect(cubit.state, ChatInitial());

      // Act - Watch chat
      cubit.watchChat('chat1');
      await Future.delayed(const Duration(milliseconds: 100));

      // Assert - Mensajes recibidos
      expect(cubit.state, isA<ChatLoaded>());

      // Act - Enviar mensaje
      cubit.sendMessage('Hello');
      await Future.delayed(const Duration(milliseconds: 100));

      // Assert - Mensaje enviado (estado puede variar)
      verify(() => mockSendMessage(any())).called(1);
    });
  });
}
```

```

test('debería manejar errores de red durante streaming', () async {
  // Arrange
  when(() => mockWatchMessages('chat1'))
    .thenAnswer((_) => Stream.value(const
      Left(NetworkFailure('Sin conexión'))));

  // Act
  cubit.watchChat('chat1');
  await Future.delayed(const Duration(milliseconds: 100));

  // Assert
  expect(cubit.state, const ChatError('Sin conexión'));
});

test('debería limpiar al cambiar de chat', () async {
  // Arrange
  final controller = StreamController<Either<Failure,
    List<Message>>>.broadcast();

  when(() => mockWatchMessages(any()))
    .thenAnswer((_) => controller.stream);

  // Act
  cubit.watchChat('chat1');
  await Future.delayed(const Duration(milliseconds: 50));
  expect(cubit.state, isA<ChatLoaded>()); // Estado del primer
    chat

  // Cambiar chat - debería limpiar el anterior
  controller.add(Right([])); // Simular nuevo chat vacío

  await Future.delayed(const Duration(milliseconds: 50));

  // El estado debería actualizarse con el nuevo chat
  // (La lógica exacta depende de la implementación)
});
});
}

```

10. Errores Comunes

Error 1: Stream no se Cancela

```

// ✗
@override
Future<void> close() {
  return super.close(); // Falta _subscription?.cancel()
}

```

Error 2: Escuchar Múltiples Veces

```
// ❌ Multiple suscripciones sin cancelar
void start() {
    _sub1 = stream.listen(...);
    _sub2 = stream.listen(...); // Duplicado!
}

// ✅ Usar share() para stream compartido
final sharedStream = stream.share();

// 0 broadcast para múltiples subscribers
final broadcastStream =
    StreamController().stream.asBroadcastStream();
```

Error 3: Tipos Incorrectos

```
// ❌ Confundir Stream<Either> con Stream<List>
Stream<Either<Failure, List<Message>>> stream = ...

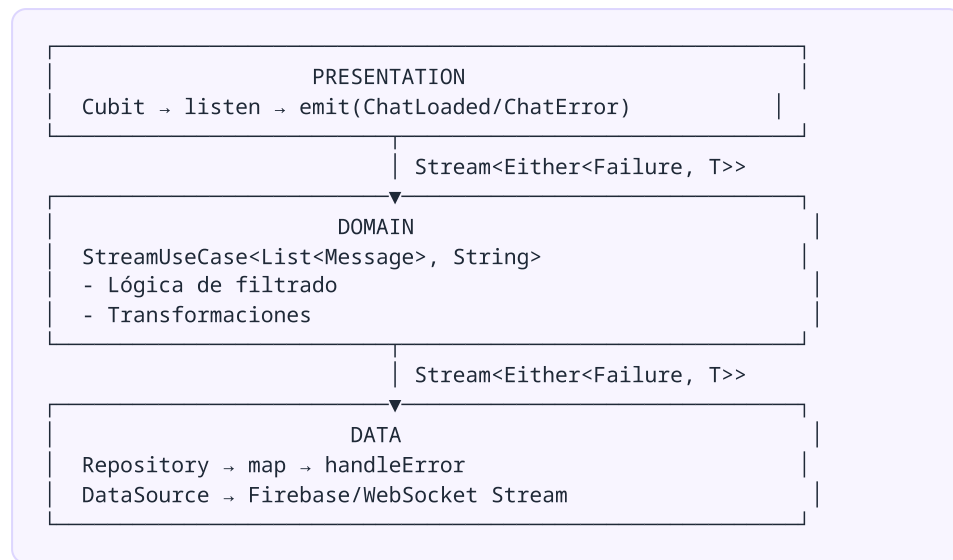
// map retorna Stream<Either<List<Transformado>>>
final mapped = stream.map((e) => e.map((msgs) => msgs.length));
//           ↑                               ↑
//           Este map es de fpdart           Este map es de Stream
```

11. Mejores Prácticas

Checklist de Streams

- ☐ ¿Cancelaste el `StreamSubscription` en `close()`?
- ☐ ¿Manejas errores del stream con `handleError`?
- ☐ ¿Usas `distinct()` para evitar duplicados?
- ☐ ¿El repository retorna `Either<Failure, T>` dentro del Stream?
- ☐ ¿Hay backpressure si los datos vienen muy rápido?

Arquitectura de Streams



12. Casos de Uso Avanzados

Combinar Múltiples Streams

```
// Combinar stream de mensajes con stream de typing
Stream<Either<Failure, ChatViewState>> watchChatState(String chatId)
{
    final messagesStream = watchMessages(chatId);
    final typingStream = watchTypingStatus(chatId);

    return messagesStream.asyncMap((messages) async {
        final typing = await typingStream.first;

        return messages.map((msgs) => ChatViewState(
            messages: msgs,
            isTyping: typing,
        ));
    });
}
```

Retry Automático

```
Stream<Either<Failure, T>> withRetry(Stream<Either<Failure, T>>
    stream) {
    return stream.handleError((error) {
        // Retry logic
        return withRetry(stream);
    });
}
```

Resumen Ejecutivo

1. StreamUseCase es el equivalente reactivo de UseCase tradicional

2. **Repository** retorna `Stream<Either<Failure, T>>` para manejar errores en el flujo
3. **DataSource** conecta con Firebase, WebSocket, o cualquier fuente de datos continuos
4. **Cubit** escucha el stream y emite estados
5. **Siempre cancelar** `StreamSubscription` en `close()`
6. **Operadores:** `map`, `filter`, `debounce`, `throttle`, `buffer` son esenciales
7. **Backpressure:** Cuando los datos llegan muy rápido, `sample`, `throttle`, o `buffer`

Siguiente nivel: Implementa un sistema de sincronización offline-first con Drift/Hive + Streams, donde los cambios locales se sincronizan automáticamente cuando hay conexión.

 **Ejemplo de chat en tiempo real con Bloc + UI completa:** [16-BLOC-CUBIT/12-ejemplo-chat-stream.md](#)
— burbujas de chat, scroll automático, indicador “escribiendo...”, `blocTest` con `Stream`.

Recursos Adicionales

- Documentación Streams Dart
- fpdart Stream utilities
- Firebase Firestore Streams
- RxJS Marbles - Diagramas interactivos de operadores

Ver también

- [11-PRUEBAS](#) — Testing de streams y cubits con `blocTest`
- [09-WORKMANAGER](#) — Tareas en segundo plano y sincronización

En el siguiente módulo

→ [05-build-runner-ecosystem.md](#) — Ecosistema completo de `build_runner`

Nivel Experto



Nivel Experto: Build Runner Ecosystem

build_runner es el motor de generación de código en Dart/Flutter. Entender cómo funciona es clave para dominar el ecosistema de codegen.

1. ¿Qué es build_runner?

build_runner es un sistema de generación de código basado en el paquete `build` de Dart. Permite ejecutar “builders” que analizan tu código fuente y generan archivos `.dart` adicionales.

1.1 El Problema que Resuelve

```
// Sin build_runner: tienes que escribir TODO manualmente
class User {
  final String id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  // Toda esta boilerplate la escribes TÚ:
  User.fromJson(Map<String, dynamic> json)
    : id = json['id'] as String,
      name = json['name'] as String,
      email = json['email'] as String;

  Map<String, dynamic> toJson() => {
    'id': id,
    'name': name,
    'email': email,
  };

  @override
  bool operator ==(Object other) =>
    identical(this, other) ||
    other is User &&
    runtimeType == other.runtimeType &&
    id == other.id &&
    name == other.name &&
    email == other.email;

  @override
  int get hashCode => Object.hash(id, name, email);
}
```

Con build_runner + generadores:

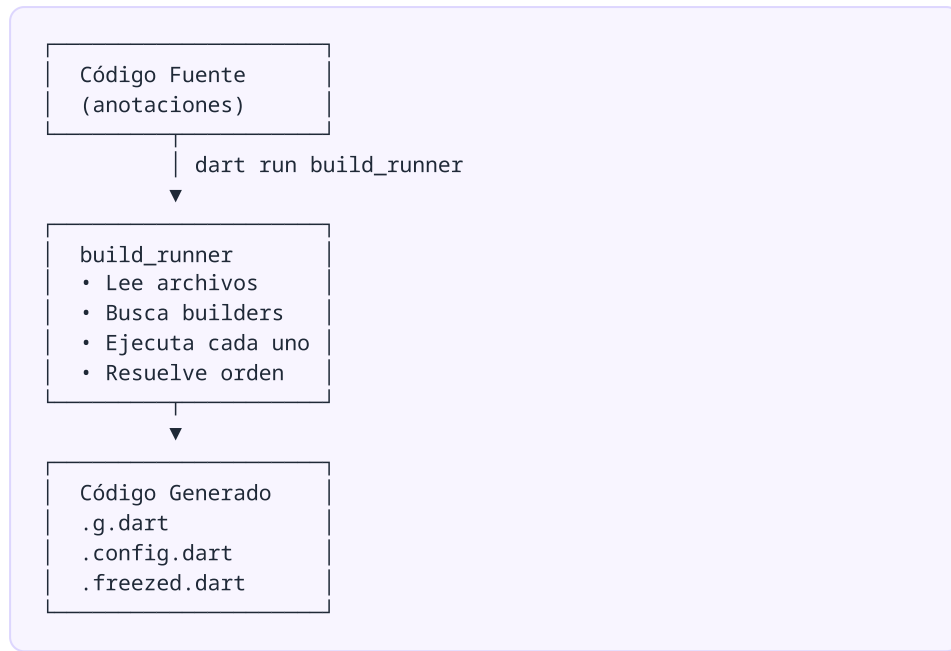
```
@JsonSerializable()
@immutable
class User {
  final String id;
  final String name;
  final String email;

  const User({
    required this.id,
    required this.name,
    required this.email,
  });

  factory User.fromJson(Map<String, dynamic> json) =>
    _$UserFromJson(json);
  Map<String, dynamic> toJson() => _$UserToJson(this);
}
```

build_runner genera automáticamente el archivo `user.g.dart` con `_$_UserFromJson` y `_$_UserToJson`.

1.2 Cómo Funciona



2. Comandos Esenciales

2.1 Comandos Principales

```
# Build único
dart run build_runner build

# Build único (elimina outputs conflictivos)
dart run build_runner build --delete-conflicting-outputs

# Modo watch: regenera automáticamente al guardar
dart run build_runner watch

# Modo watch con limpieza
dart run build_runner watch --delete-conflicting-outputs

# Limpiar caché de generación
dart run build_runner clean
```

2.2 Cuando Usar Cada Uno

Comando	Cuándo Usarlo
<code>build --delete-conflicting-outputs</code>	CI/CD, setup inicial, después de pull
<code>watch --delete-conflicting-outputs</code>	Desarrollo activo (recomendado)
<code>build</code>	Si sabes que no hay conflictos
<code>clean</code>	Cuando el generador se comporta extraño

2.3 Integración con Makefile

```
# Makefile

MOBILE_DIR = apps/mobile

.PHONY: codegen codegen-watch clean

codegen:
    @cd $(MOBILE_DIR) && dart run build_runner build --delete-conflicting-outputs

codegen-watch:
    @cd $(MOBILE_DIR) && dart run build_runner watch --delete-conflicting-outputs

clean:
    @cd $(MOBILE_DIR) && dart run build_runner clean
```

3. Builders: El Corazón del Sistema

3.1 ¿Qué es un Builder?

Un builder es un paquete que sabe cómo generar código. Ejemplos:

Builder	Paquete	Genera
<code>json_serializable</code>	<code>json_serializable</code>	<code>.g.dart</code> con <code>fromJson/toJson</code>
<code>freezed</code>	<code>freezed</code>	<code>.freezed.dart</code> con <code>data classes</code>
<code>injectable_generator</code>	<code>injectable_generator</code>	<code>*.config.dart</code> con <code>DI</code>
<code>retrofit_generator</code>	<code>retrofit</code>	<code>.g.dart</code> con <code>clientes HTTP</code>
<code>isar_community_generator</code>	<code>isar</code>	<code>.g.dart</code> con <code>código Isar</code>

3.2 Cómo Dart Encuentra los Builders

Los builders se registran en `pubspec.yaml` del paquete que los define:

```
# Ejemplo: pubspec.yaml de json_serializable
environment:
  sdk: ">=3.0.0 <4.0.0"

builders:
  json_serializable:
    import: "package:json_serializable/builder.dart"
    builder_factories: ["jsonSerializable"]
    build_extensions: {".dart": [".json_serializable.g.part"]}
    auto_apply: dependents
    build_to: cache
```

Cuando ejecutas `build_runner`, este escanea todos tus paquetes, encuentra los builders registrados, y los ejecuta en el orden correcto.

4. Archivo build.yaml

4.1 Configuración Básica

```
# build.yaml

targets:
  $default:
    builders:
      json_serializable:
        options:
          explicit_to_json: true
          include_if_null: false
          create_factory: true
```

4.2 Configuración por Builder

```
targets:
  $default:
    builders:
      json_serializable:
        options:
          explicit_to_json: true
          field_rename: snake
          checked: true
          create_to_json: true
          any_map: false

      injectable_generator: injectable_builder:
        options:
          auto_register: true
          relative_paths: true

      freezed:
        options:
          union_key: type
          union_value_case: pascal
```

4.3 Opciones Comunes de json_serializable

Opción	Valores	Descripción
<code>explicit_to_json</code>	true/false	Genera <code>toJson</code> explícito
<code>include_if_null</code>	true/false	Incluir campos null en JSON
<code>field_rename</code>	none, snake, kebab, pascal	Cómo convertir nombres
<code>checked</code>	true/false	Validar tipos en deserialización
<code>any_map</code>	true/false	Aceptar Map en lugar de solo Map<String, dynamic>

4.4 Configuración para Monorepo

```
# build.yaml en monorepo con múltiples paquetes
targets:
  $default:
    builders:
      json_serializable:
        generate_for:
          - "lib/**_models.dart"
          - "lib/**/*_dto.dart"
        options:
          explicit_to_json: true

# Excluir tests de generación
test:
  builders:
    json_serializable:
      generate_for:
        exclude:
          - "test/**"
```

5. Archivos Generados y Control de Versiones

5.1 Convención de Nombres

Archivo	Contenido
user.g.dart	Código generado (json, retrofit, Isar)
user.freezed.dart	Clases inmutables con freezed
user.g.part	Partial (usado internamente por freezed)
injection_container.config.dart	DI generado con injectable

5.2 ¿Comitar .g.dart o No?

Regla general: SÍ se comitan.

Razones: - Los generadores pueden tener breaking changes entre versiones - Evita que cada dev tenga que ejecutar codegen al clonar - CI/CD no necesita build_runner para analizar - Los `.g.dart` suelen ser estables y legibles

Excepción: No comitar si: - El equipo usa CI que siempre ejecuta codegen - Usas `build.summary` o `build.cache` (no comitar estos)

5.3 analysis_options.yaml

```
analyzer:
  exclude:
    - "**/*.g.dart"
    - "**/*.freezed.dart"
    - "**/*.config.dart"
    - lib/l10n/gen/*
```

5.4 .gitignore

```
# Opcional: solo si regeneras siempre en CI
# **/*.g.dart

# Cache de build_runner (NUNCA comitar)
.dart_tool/
build/
```

6. Solución de Problemas

6.1 Error: Conflicting Outputs

```
[SEVERE] Conflict: outputs share names
```

Causa: Dos builders generan el mismo archivo.

Solución:

```
# build.yaml
targets:
  $default:
    builders:
      builder_a:
        generate_for:
          - "lib/feature_a/**"
      builder_b:
        generate_for:
          - "lib/feature_b/**"
```

6.2 Error: Stack Overflow / Memoria

```
FATAL: Error during build: Stack Overflow
```

Causa: Dependencia circular entre builders o archivos.

Solución: - Separar modelos en archivos más pequeños -
Revisar imports circulares - Aumentar memoria: `dart run build_runner build --define=build_runner=memory=2048`

6.3 Error: Versiones Incompatibles

```
[SEVERE] json_serializable requires builder >=3.0.0
```

Causa: Versiones de generadores incompatibles.

Solución:

```
# Actualizar todo
dart pub upgrade

# Si persiste: verificar constraints en pubspec.yaml
dart pub deps
```

6.4 Error: Build Lento

Causas comunes y soluciones:

Problema	Solución
Muchos archivos sin anotaciones	Usar <code>generate_for</code> en <code>build.yaml</code>
Builds desde cero cada vez	Usar <code>watch</code> en lugar de <code>build</code>
Demasiados builders	Deshabilitar los que no usas
Archivos muy grandes	Dividir en archivos más pequeños

7. Flujo de Trabajo Recomendado

7.1 Desarrollo Diario

```
# Terminal 1: Modo watch
dart run build_runner watch --delete-conflicting-outputs

# Terminal 2: Tu IDE normal
flutter run
```

7.2 CI/CD

```
# .github/workflows/ci.yml
jobs:
  build:
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
      - run: dart pub get
      - run: dart run build_runner build --delete-conflicting-outputs
      - run: flutter analyze
      - run: flutter test
```

7.3 Setup Inicial del Proyecto

```
# 1. Agregar dependencias
flutter pub add json_annotation freezed_annotation
flutter pub add dev:build_runner dev:json_serializable dev:freezed

# 2. Crear build.yaml
touch build.yaml

# 3. Ejecutar codegen
dart run build_runner build --delete-conflicting-outputs

# 4. Verificar que se generaron los archivos
git status
```

8. Errores Comunes y Debugging

8.1 Logging Detallado

```
# Modo verbose
dart run build_runner build --verbose

# Guardar log
dart run build_runner build 2>&1 | tee build_runner.log
```

8.2 Verificar Qué Builders Están Registrados

```
# Listar builders disponibles
dart run build_runner build --list-builders
```

8.3 Limpiar y Reconstruir

```
# Cuando todo falla
dart run build_runner clean
rm -rf .dart_tool
dart pub get
dart run build_runner build --delete-conflicting-outputs
```

9. Comparativa: build_runner vs Alternativas

Herramienta	Enfoque	Ventajas	Desventajas
build_runner	Análisis de código	Potente, extensible, estándar	Lento en proyectos grandes
macros (próximamente)	Compile-time	Rápido, integrado en Dart	Aún no estable (Dart 3.7+)
manual	Script propio	Control total	Mucho trabajo
custom_generators	Template-based	Rápido para casos simples	Limitado

Nota: Dart está introduciendo `macros` (experimental desde Dart 3.5+) que eventualmente reemplazarán a `build_runner` para muchos casos. Sin embargo, `build_runner` seguirá siendo el estándar por varios años.

10. Crear un Builder Personalizado

Para equipos que necesitan generación específica:

```
# pubspec.yaml de tu builder
name: my_custom_builder
version: 1.0.0

environment:
  sdk: ">=3.0.0 <4.0.0"

dependencies:
  build: ^2.4.0
  source_gen: ^1.5.0
  analyzer: ^6.0.0

builders:
  my_builder:
    import: "package:my_custom_builder/builder.dart"
    builder_factories: ["myBuilder"]
    build_extensions: {".dart": [".my_builder.g.part"]}
    auto_apply: dependents
    build_to: cache
```

```
// builder.dart
import 'package:build/build.dart';
import 'package:source_gen/source_gen.dart';

Builder myBuilder(BuilderOptions options) {
  return SharedPartBuilder(
    [MyGenerator()],
    'my_builder',
  );
}

class MyGenerator extends Generator {
  @override
  String generate(LibraryReader library, BuildStep buildStep) {
    // Analizar código y generar
    final classes = library.classes;
    // ... lógica de generación
    return generatedCode;
  }
}
```

11. Integración con Isar (del proyecto real)

En el monorepo, Isar es el único consumidor de build_runner:

```
# pubspec.yaml
dev_dependencies:
  build_runner: ^2.4.7
  isar_community_generator: ^3.3.2
```

```
// Modelo Isar con @Collection
import 'package:isar/isar.dart';

part 'cached_user.g.dart';

@Collection()
class CachedUser {
  @Index(unique: true)
  late String userId;

  late String name;
  late String email;

  Id id = Isar.autoIncrement;
}
```

El generador produce `cached_user.g.dart` con todo el código de serialización y almacenamiento Isar.

Flujo de Trabajo con Isar

```
# 1. Crear/modificar modelo con @Collection()
# 2. Ejecutar código
dart run build_runner build --delete-conflicting-outputs
# 3. Usar el modelo generado
```


12. Resumen Ejecutivo

1. **build_runner** es el motor de codegen estándar en Dart/Flutter
 2. **build.yaml** configura cómo se ejecutan los builders
 3. **--delete-conflicting-outputs** es tu aliado (úsalo siempre)
 4. Modo **watch** para desarrollo, **build** para CI/CD
 5. Los **.g.dart** se comitan en el repo
 6. **analysis_options.yaml** debe excluir archivos generados
 7. **build_runner** + **Makefile** = flujo de trabajo predecible
-

Recursos Adicionales

- **build_runner** en pub.dev
 - **build_config** documentation
 - **source_gen** package
 - Isar Community Generator
-

Ver también

- **06-json-serializable-freezed.md** — Serialización con **json_serializable** + **Freezed**
 - **07-retrofit-api-client.md** — Clientes HTTP con **Retrofit**
-

En el siguiente módulo

→ **06-json-serializable-freezed.md** — **json_serializable** y **Freezed** para data classes

Nivel Experto



Nivel Experto: json_serializable y Freezed

La combinación de json_serializable + freezed elimina TODO el boilerplate de modelos: serialización, inmutabilidad, igualdad, copia, y pattern matching.

1. json_serializable: Serialización Automática

1.1 Configuración

```
# pubspec.yaml
dependencies:
  json_annotation: ^4.9.0
  freezed_annotation: ^2.4.0

dev_dependencies:
  build_runner: ^2.4.8
  json_serializable: ^6.8.0
  freezed: ^2.5.0
```

```
# build.yaml
targets:
  $default:
    builders:
      json_serializable:
        options:
          explicit_to_json: true
          field_rename: snake
          checked: true
```

1.2 Uso Básico

```
import 'package:json_annotation/json_annotation.dart';

part 'user.g.dart';

@JsonSerializable()
class User {
  final String id;
  final String name;
  final String email;
  final DateTime? createdAt;

  const User({
    required this.id,
    required this.name,
    required this.email,
    this.createdAt,
  });

  factory User.fromJson(Map<String, dynamic> json) =>
    _$UserFromJson(json);
  Map<String, dynamic> toJson() => _$UserToJson(this);
}
```

Genera: `user.g.dart` con `_UserFromJson` y `_UserToJson`.

1.3 Anotaciones

```
@JsonSerializable()
class Product {
    // Campos con nombre distinto en JSON
    @JsonKey(name: 'product_id')
    final String id;

    // Ignorar campo
    @JsonKey(includeFromJson: false, includeToJson: false)
    final String? cacheOnly;

    // Valor por defecto si falta en JSON
    @JsonKey(defaultValue: 0)
    final int stock;

    // Campo requerido
    @JsonKey(required: true)
    final double price;

    // Ignorar si null
    @JsonKey(includeIfNull: false)
    final String? description;

    // Convertir entre tipos
    @JsonKey(fromJson: _dateFromJson, toJson: _dateToJson)
    final DateTime? updatedAt;

    const Product({...});

    factory Product.fromJson(Map<String, dynamic> json) =>
        _$ProductFromJson(json);
    Map<String, dynamic> toJson() => _$ProductToJson(this);

    static DateTime? _dateFromJson(int? timestamp) =>
        timestamp != null ?
            DateTime.fromMillisecondsSinceEpoch(timestamp) : null;

    static int? _dateToJson(DateTime? date) =>
        date?.millisecondsSinceEpoch;
}
```

1.4 Configuración Global vs Local

```
// Global (build.yaml): field_rename: snake
// Local (override):
@JsonSerializable(fieldRename: FieldRename.none)
class LocalModel {
    final String fullName; // en JSON: "fullName", no "full_name"
}
```

1.5 Enums con json_serializable

```
@JsonEnum(valueField: 'value')
enum Status {
  active('active'),
  inactive('inactive'),
  pending('pending');

  const Status(this.value);
  final String value;
}

@JsonSerializable()
class Order {
  final Status status;

  @JsonKey(name: 'order_date')
  final DateTime orderDate;

  const Order({required this.status, required this.orderDate});

  factory Order.fromJson(Map<String, dynamic> json) =>
    _$OrderFromJson(json);
  Map<String, dynamic> toJson() => _$OrderToJson(this);
}
```

1.6 Anidación y Listas

```
@JsonSerializable()
class Category {
  final String id;
  final String name;
  final List<Product> products; // Se serializa automáticamente

  const Category({...});
  factory Category.fromJson(Map<String, dynamic> json) =>
    _$CategoryFromJson(json);
  Map<String, dynamic> toJson() => _$CategoryToJson(this);
}
```

1.7 Heredar Serialización

```
@JsonSerializable()
class BaseEntity {
    final String id;
    final DateTime createdAt;

    const BaseEntity({required this.id, required this.createdAt});
}

@JsonSerializable()
class ExtendedEntity extends BaseEntity {
    final String extraField;

    const ExtendedEntity({
        required super.id,
        required super.createdAt,
        required this.extraField,
    });

    factory ExtendedEntity.fromJson(Map<String, dynamic> json) =>
        _$ExtendedEntityFromJson(json);

    @override
    Map<String, dynamic> toJson() => _$ExtendedEntityToJson(this);
}
```

2. Freezed: Inmutabilidad y Data Classes

2.1 El Problema que Resuelve

```
// Sin freezed: 30+ líneas de boilerplate por clase
class User {
    final String id;
    final String name;

    const User({required this.id, required this.name});

    // copyWith
    User copyWith({String? id, String? name}) => User(
        id: id ?? this.id,
        name: name ?? this.name,
    );

    // == operator
    @override
    bool operator ==(Object other) =>
        identical(this, other) ||
        other is User && runtimeType == other.runtimeType && id ==
            other.id && name == other.name;

    @override
    int get hashCode => Object.hash(id, name);

    @override
    String toString() => 'User(id: $id, name: $name)';
}
```

```
// Con freezed: ~10 líneas
@freezed
class User with _$User {
  const factory User({
    required String id,
    required String name,
  }) = _User;
}
```

Genera automáticamente: `copyWith`, `==`, `hashCode`, `toString`, y más.

2.2 Instalación y Configuración

```
# pubspec.yaml
dependencies:
  freezed_annotation: ^2.4.0
  json_annotation: ^4.9.0

dev_dependencies:
  build_runner: ^2.4.8
  freezed: ^2.5.0
  json_serializable: ^6.8.0
```

```
# build.yaml
targets:
  $default:
    builders:
      freezed:
        options:
          union_key: type
          union_value_case: pascal
```

2.3 Uso Básico

```
import 'package:freezed_annotation/freezed_annotation.dart';

part 'user.freezed.dart';
part 'user.g.dart';

@freezed
class User with _$User {
  const factory User({
    required String id,
    required String name,
    @Default(0) int age,           // Valor por defecto
    String? email,
  }) = _User;

  factory User.fromJson(Map<String, dynamic> json) =>
    _$UserFromJson(json);
}
```

Lo que genera freezed:

Método	Generado	Ejemplo
copyWith	✓	user.copyWith(name: 'new')
== / hashCode	✓	Comparación por valor
toString	✓	User(id: 1, name: Test)
Pattern matching	✓	user.when(...) / user.map(...)
Getters	✓	user.name

2.4 Uniones Discriminadas (Sealed Unions)

El caso de uso más poderoso de freezed: modelar estados.

```
@freezed
class UserState with _$UserState {
  const factory UserState.initial() = Initial;
  const factory UserState.loading() = Loading;
  const factory UserState.loaded({
    required User user,
  }) = Loaded;
  const factory UserState.error(String message) = Error;
}
```

Pattern matching exhaustivo:

```
void handleState(UserState state) {
  state.when(
    initial: () => print('Esperando...'),
    loading: () => showSpinner(),
    loaded: (user) => showUser(user),
    error: (message) => showError(message),
    // El compilador OBLIGA a cubrir todos los casos
  );
}
```

when obliga a manejar TODOS los casos. Si agregas un nuevo estado, el compilador lanza error en cada **when**.

2.5 Patrones de Freezed

```
@freezed
class AsyncValue<T> with _$AsyncValue<T> {
  // Genéricos
  const factory AsyncValue.loading() = AsyncLoading<T>;
  const factory AsyncValue.data(T value) = AsyncData<T>;
  const factory AsyncValue.error(String message) = AsyncError<T>;
}

// Uso
final value = AsyncValue<int>.data(42);
final result = value.when(
  loading: () => null,
  data: (v) => v.toString(),
  error: (msg) => 'Error: $msg',
);
```


2.6 copyWith en Profundidad

```
@freezed
class Order with _Order {
  const factory Order({
    required String id,
    required List<Item> items,
    String? notes,
  }) = _Order;
}

final order = Order(id: '1', items: [Item(name: 'A')]);

// Copy básico
order.copyWith(notes: 'Urgente');

// Copy anidado (con data classes anidadas)
order.copyWith.items([Item(name: 'B')]);

// Copy con transformación
order.copyWith(items: (items) => [...items, Item(name: 'C')]);
```

2.7 Métodos Personalizados

```
@freezed
class Product with _Product {
  const factory Product({
    required String name,
    required double price,
    @Default(0) double discount,
  }) = _Product;

  const Product.(); // Constructor privado para métodos

  // Métodos de instancia
  double get finalPrice => price * (1 - discount / 100);

  // Getters
  bool get hasDiscount => discount > 0;
}
```

2.8 Freezed + json_serializable

```
@freezed
class ApiResponse<T> with _ApiResponse<T> {
  const factory ApiResponse.success(T data) = ApiSuccess<T>;
  const factory ApiResponse.error({
    required int code,
    required String message,
  }) = ApiError<T>;

  factory ApiResponse.fromJson(
    Map<String, dynamic> json,
    T Function(Object?) fromJsonT,
  ) => _ApiResponseFromJson(json, fromJsonT);
}
```

3. json_serializable + freezed + fpdart: Pipeline

Completo

3.1 Modelo de Dominio

```
// domain/entities/user.dart
@freezed
class User with _$User {
  const factory User({
    required String id,
    required String name,
    required String email,
  }) = _User;
}
```

3.2 Modelo de Datos (DTO)

```
// data/models/user_dto.dart
@freezed
class UserDto with _$UserDto {
  const factory UserDto({
    @JsonKey(name: 'user_id') required String id,
    @JsonKey(name: 'full_name') required String name,
    required String email,
  }) = _UserDto;

  factory UserDto.fromJson(Map<String, dynamic> json) =>
    _UserDtoFromJson(json);

  factory UserDto.fromDomain(User user) => UserDto(
    id: user.id,
    name: user.name,
    email: user.email,
  );

  User toDomain() => User(
    id: id,
    name: name,
    email: email,
  );
}
```

3.3 Mapper Manual

```
// data/repositories/user_repository_impl.dart
@lazySingleton(as: UserRepository)
class UserRepositoryImpl implements UserRepository {
  final ApiClient apiClient;

  UserRepositoryImpl(this.apiClient);

  @override
  Future<Either<Failure, User>> getUser(String id) async {
    try {
      final response = await apiClient.getUser(id);
      final user = response.toDomain(); // DTO -> Domain
      return Right(user);
    } catch (e) {
      return Left(ServerFailure(e.toString()));
    }
  }
}
```

4. Buenas Prácticas

4.1 Separación DTO / Domain

```
lib/
├── features/
│   └── users/
│       ├── data/
│       │   ├── models/
│       │   │   ├── user_dto.dart      # freezed + json_serializable
│       │   │   ├── user_dto.g.dart   # GENERADO
│       │   │   └── user_dto.freezed.dart # GENERADO
│       │   ├── datasources/
│       │   └── repositories/
│       └── domain/
│           ├── entities/
│           │   └── user.dart           # freezed (sin json)
│           ├── repositories/
│           └── usecases/
```

Por qué separar:

- Los DTOs cambian con la API, las entidades no
- La capa de dominio no debe depender de JSON
- Cada capa puede evolucionar independientemente

4.2 Evitar Anidación Excesiva

```
// ❌ Difícil de leer y mantener
@freezed
class ComplexResponse with _$ComplexResponse {
  const factory ComplexResponse({
    @Default([]) List<@Default([]) List<String>> matrix,
    Map<String, @Default(0) int>? counts,
  }) = _ComplexResponse;
}

// ✅ Separar en tipos concretos
@freezed
class MatrixRow with _$MatrixRow {
  const factory MatrixRow({
    @Default([]) List<String> values,
  }) = _MatrixRow;
}
```

4.3 Límites de Generación

```
# build.yaml: evitar generar para tests y archivos específicos
targets:
  $default:
    builders:
      json_serializable:
        generate_for:
          - "lib/**"
        options:
          explicit_to_json: true
```

5. Conclusión

Herramienta	Para qué	Alternativa sin codegen
json_serializable	Serialización JSON	Escribir fromJson/toJson manual
freezed	Data classes inmutables	copyWith/==/hashCode manual
Ambos juntos	Pipeline completo	~100 líneas de boilerplate por clase

Regla de oro: Si un modelo tiene 3+ campos, usa freezed + json_serializable. El tiempo que te ahorras en debugging de `==` y `copyWith` justifica la configuración inicial.

Recursos Adicionales

- json_serializable pub.dev
- freezed pub.dev
- Freezed Documentation

- [JSON Serialization Guide](#)
-

Ver también

- [05-build-runner-ecosystem.md](#) — Ecosistema de build_runner
 - [13-MANUAL-JSON](#) — Serialización manual alternativa
-

En el siguiente módulo

→ [07-retrofit-api-client.md](#) — Clientes HTTP generados con Retrofit

Nivel Experto

Nivel Experto: Retrofit - Clientes HTTP Generados

Retrofit genera automáticamente clientes HTTP a partir de anotaciones. Es el estándar para consumir APIs REST en Flutter profesional.

1. ¿Por qué Retrofit?

1.1 Sin Retrofit (Manual)

```
class UserApiClient {
  final http.Client client;
  final String baseUrl;

  UserApiClient({required this.client, required this.baseUrl});

  Future<UserDto> getUser(String id) async {
    final response = await client.get(
      Uri.parse('$baseUrl/users/$id'),
      headers: {'Content-Type': 'application/json'},
    );

    if (response.statusCode == 200) {
      return UserDto.fromJson(jsonDecode(response.body));
    } else if (response.statusCode == 404) {
      throw NotFoundException('User not found');
    } else {
      throw ApiException('Error: ${response.statusCode}');
    }
  }

  // 10+ métodos más... todo boilerplate repetitivo
}
```

Problemas: - Cada endpoint = repetir URI, headers, parsing - Errores tipográficos en URIs (solo se detectan en runtime) - El manejo de errores se vuelve inconsistente - Mucho código duplicado

1.2 Con Retrofit

```
@RestApi(baseUrl: 'https://api.example.com')
abstract class UserApiClient {
    factory UserApiClient(Dio dio, {String baseUrl}) = _UserApiClient;

    @GET('/users/{id}')
    Future<UserDto> getUser(@Path('id') String id);

    @GET('/users')
    Future<List<UserDto>> getUsers();

    @POST('/users')
    Future<UserDto> createUser(@Body() CreateUserDto dto);
}
```

Genera automáticamente: el `_UserApiClient` con toda la implementación.

2. Configuración

2.1 Dependencias

```
# pubspec.yaml
dependencies:
  dio: ^5.4.0
  retrofit: ^4.1.0
  json_annotation: ^4.9.0

dev_dependencies:
  build_runner: ^2.4.8
  retrofit_generator: ^8.1.0
  json_serializable: ^6.8.0
```

2.2 build.yaml

```
# build.yaml
targets:
  $default:
    builders:
      retrofit_generator:
        options:
          nullable: true
          named_parameters: false
```

2.3 Cliente Básico

```
import 'package:dio/dio.dart';
import 'package:retrofit/retrofit.dart';
import 'package:json_annotation/json_annotation.dart';

part 'api_client.g.dart';

@RestApi(baseUrl: 'https://api.example.com/v1')
abstract class ApiClient {
  factory ApiClient(Dio dio, {String baseUrl}) = _ApiClient;
}
```

```
// main.dart
final dio = Dio(BaseOptions(
  connectTimeout: const Duration(seconds: 10),
  receiveTimeout: const Duration(seconds: 10),
));

final api = ApiClient(dio);
```

3. Anotaciones

3.1 Métodos HTTP

Anotación	HTTP	Ejemplo
@GET('path')	GET	@GET('/users')
@POST('path')	POST	@POST('/users')
@PUT('path')	PUT	@PUT('/users/{id}')
@PATCH('path')	PATCH	@PATCH('/users/{id}')
@DELETE('path')	DELETE	@DELETE('/users/{id}')
@HEAD('path')	HEAD	@HEAD('/health')
@HTTP(method: 'OPTIONS', path: '/users')	Custom	Para métodos no estándar

3.2 Parámetros

Anotación	Uso	Descripción
@Path('id')	@GET('/users/{id}')	Variable en URL
@Query('page')	@GET('/users')	Query param <code>?page=1</code>
@QueryMap()	@GET('/users')	Múltiples query params
@Body()	@POST('/users')	Cuerpo de la petición
@Header('Authorization')	@GET('/users')	Header específico
@Headers({'X-API-Key': 'value'})	@GET('/users')	Múltiples headers
@Field('name')	@POST('/users')	Form field (para FormData)
@Part()	@POST('/upload')	Parte de multipart

3.3 Ejemplos Completos

```
@RestApi(baseUrl: 'https://api.example.com/v1')
abstract class ApiClient {
    factory ApiClient(Dio dio, {String baseUrl}) = _ApiClient;

    // Path param
    @GET('/users/{id}')
    Future<UserDto> getUser(@Path('id') String id);

    // Query params
    @GET('/users')
    Future<List<UserDto>> getUsers({
        @Query('page') int? page,
        @Query('limit') int? limit,
        @Query('search') String? query,
    });

    // Query map
    @GET('/users')
    Future<List<UserDto>> searchUsers(@QueryMap() Map<String, dynamic>
        filters);

    // POST con body
    @POST('/users')
    Future<UserDto> createUser(@Body() CreateUserDto user);

    // PUT con path + body
    @PUT('/users/{id}')
    Future<UserDto> updateUser(
        @Path('id') String id,
        @Body() UpdateUserDto user,
    );

    // DELETE
    @DELETE('/users/{id}')
    Future<void> deleteUser(@Path('id') String id);

    // Headers estáticos
    @GET('/admin/users')
    @Headers({
        'X-Admin-Role': 'true',
        'X-Request-Source': 'mobile',
    })
    Future<List<UserDto>> getAdminUsers();

    // Header dinámico
    @GET('/users/me')
    Future<UserDto> getCurrentUser(
        @Header('Authorization') String token,
    );

    // Form fields
    @POST('/auth/login')
    Future<AuthTokenDto> login(
        @Field('email') String email,
        @Field('password') String password,
    );

    // Multipart upload
    @POST('/files/upload')
    @MultiPart()
    Future<FileDto> uploadFile(@Part() FileParam file);

    // Respuesta sin procesar
    @GET('/raw')
    Future<HttpResponse<String>> getRawResponse();
}
```

4. Manejo de Errores

4.1 Interceptors de Dio

```
// lib/core/network/error_interceptor.dart
class ErrorInterceptor extends Interceptor {
  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    switch (err.type) {
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.sendTimeout:
      case DioExceptionType.receiveTimeout:
        throw NetworkTimeoutException();
      case DioExceptionType.badResponse:
        switch (err.response?.statusCode) {
          case 400:
            throw BadRequestException(err.response?.data['message']);
          case 401:
            throw UnauthorizedException();
          case 403:
            throw ForbiddenException();
          case 404:
            throw NotFoundException();
          case 500:
            throw ServerException();
        }
      case DioExceptionType.cancel:
        throw RequestCancelledException();
      default:
        throw NetworkException(err.message);
    }
  }
}
```

4.2 Wrapper con Either

```
class ApiClientWrapper {
  final ApiClient apiClient;

  ApiClientWrapper(this.apiClient);

  Future<Either<Failure, User>> getUser(String id) async {
    try {
      final dto = await apiClient.getUser(id);
      return Right(dto.toDomain());
    } on NotFoundException {
      return Left(UserNotFoundFailure());
    } on UnauthorizedException {
      return Left(AuthFailure());
    } on ServerException {
      return Left(ServerFailure('Server error'));
    } on NetworkException {
      return Left(NetworkFailure());
    }
  }
}
```

4.3 Logging Interceptor

```
class LoggingInterceptor extends Interceptor {  
  @override  
  void onRequest(RequestOptions options, RequestInterceptorHandler  
    handler) {  
    print('${options.method}: ${options.path}');  
    print('Headers: ${options.headers}');  
    print('Body: ${options.data}');  
    handler.next(options);  
  }  
  
  @override  
  void onResponse(Response response, ResponseInterceptorHandler  
    handler) {  
    print('${response.statusCode}: ${response.requestOptions.path}');  
    handler.next(response);  
  }  
  
  @override  
  void onError(DioException err, ErrorInterceptorHandler handler) {  
    print('ERROR: ${err.type} - ${err.message}');  
    handler.next(err);  
  }  
}
```

5. Configuración Avanzada de Dio

5.1 Cliente Configurable

```
class ApiService {  
  late final Dio _dio;  
  late final ApiClient _apiClient;  
  
  ApiService({required String baseUrl, required String? token}) {  
    _dio = Dio(BaseOptions(  
      baseUrl: baseUrl,  
      connectTimeout: const Duration(seconds: 15),  
      receiveTimeout: const Duration(seconds: 15),  
      headers: {  
        'Content-Type': 'application/json',  
        'Accept': 'application/json',  
      },  
    ));  
  
    // Interceptors  
    _dio.interceptors.addAll([  
      AuthInterceptor(token: token),  
      LoggingInterceptor(),  
      ErrorInterceptor(),  
      RetryInterceptor(dio: _dio),  
    ]);  
  
    _apiClient = ApiClient(_dio, baseUrl: baseUrl);  
  }  
}
```

5.2 Interceptor de Autenticación

```
class AuthInterceptor extends Interceptor {
  String? _token;

  AuthInterceptor({String? token}) : _token = token;

  void updateToken(String token) => _token = token;

  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler
    handler) {
    if (_token != null) {
      options.headers['Authorization'] = 'Bearer $_token';
    }
    handler.next(options);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    if (err.response?.statusCode == 401) {
      // Token expirado
      _onTokenExpired();
    }
    handler.next(err);
  }

  void _onTokenExpired() {
    // Emitir evento para refrescar token
    // o redirigir al login
  }
}
```

5.3 Retry Interceptor

```
class RetryInterceptor extends Interceptor {
  final Dio dio;
  final int maxRetries;
  final Duration retryDelay;

  RetryInterceptor({
    required this.dio,
    this.maxRetries = 3,
    this.retryDelay = const Duration(seconds: 2),
  });

  @override
  void onError(DioException err, ErrorInterceptorHandler handler)
    async {
    if (_shouldRetry(err)) {
      final retries = (err.requestOptions.extra['retries'] as int?)
        ?? 0;
      if (retries < maxRetries) {
        await Future.delayed(retryDelay * (retries + 1));
        err.requestOptions.extra['retries'] = retries + 1;
        try {
          final response = await dio.fetch(err.requestOptions);
          handler.resolve(response);
          return;
        } catch (e) {
          // Continuar con el error original si falla
        }
      }
    }
    handler.next(err);
  }

  bool _shouldRetry(DioException err) {
    return err.type == DioExceptionType.connectionTimeout ||
      err.type == DioExceptionType.receiveTimeout ||
      err.type == DioExceptionType.connectionError ||
      (err.response?.statusCode ?? 0) >= 500;
  }
}
```

6. Integración con Clean Architecture

6.1 Estructura

```
lib/
├── core/
│   └── network/
│       ├── api_client.dart           # Anotaciones @RestApi
│       ├── api_client.g.dart        # GENERADO
│       └── interceptors/
│           ├── auth_interceptor.dart
│           ├── error_interceptor.dart
│           ├── logging_interceptor.dart
│           └── retry_interceptor.dart
│       └── api_service.dart          # Configuración de Dio
└── features/
    └── users/
        ├── data/
        │   ├── models/
        │   │   └── user_dto.dart
        │   ├── datasources/
        │   │   └── user_remote_datasource.dart # Usa api_client
        │   └── repositories/
        │       └── user_repository_impl.dart
        └── domain/...
```

6.2 Inyección con Injectable

```
// core/di/modules/network_module.dart
@module
abstract class NetworkModule {
  @lazySingleton
  Dio get dio => Dio(BaseOptions(
    connectTimeout: const Duration(seconds: 15),
    receiveTimeout: const Duration(seconds: 15),
  ));

  @lazySingleton
  ApiClient get apiClient => ApiClient(dio);

  @lazySingleton
  AuthInterceptor get authInterceptor => AuthInterceptor();
}
```

6.3 DataSource con Retrofit

```
// features/users/data/datasources/user_remote_datasource.dart
@lazySingleton
class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  final ApiClient apiClient;

  UserRemoteDataSourceImpl(this.apiClient);

  @override
  Future<UserDto> getUser(String id) async {
    return apiClient.getUser(id);
  }

  @override
  Future<List<UserDto>> getUsers({int? page, int? limit}) async {
    return apiClient.getUsers(page: page, limit: limit);
  }
}
```

7. Testing con Retrofit

7.1 Mock de ApiClient

```
@mock
abstract class MockApiClient extends Mock implements ApiClient {}

void main() {
  late MockApiClient mockApiClient;
  late UserRemoteDataSource dataSource;

  setUp(() {
    mockApiClient = MockApiClient();
    dataSource = UserRemoteDataSourceImpl(mockApiClient);
  });

  test('getUser returns UserDto', () async {
    const userDto = UserDto(id: '1', name: 'Test', email:
      'test@test.com');

    when(() => mockApiClient.getUser('1')).thenAnswer((_) async =>
      userDto);

    final result = await dataSource.getUser('1');

    expect(result, equals(userDto));
    verify(() => mockApiClient.getUser('1')).called(1);
  });
}
```

7.2 Mock de Dio

```
void main() {
  late Dio mockDio;
  late ApiClient apiClient;

  setUp(() {
    mockDio = MockDio();
    apiClient = ApiClient(mockDio);
  });

  test('getUser handles 404', () async {
    when(() => mockDio.get<dynamic>('/users/999'))
      .thenThrow(DioException(
        requestOptions: RequestOptions(path: '/users/999'),
        response: Response(
          statusCode: 404,
          requestOptions: RequestOptions(path: '/users/999'),
        ),
      ));

    expect(
      () => apiClient.getUser('999'),
      throwsA(isA<DioException>()),
    );
  });
}
```

8. Buenas Prácticas

8.1 Versionado de API

```
@RestApi(baseUrl: 'https://api.example.com')
abstract class ApiV1 {
  factory ApiV1(Dio dio) = _ApiV1;

  @GET('/v1/users')
  Future<List<UserDto>> getUsers();
}

@RestApi(baseUrl: 'https://api.example.com')
abstract class ApiV2 {
  factory ApiV2(Dio dio) = _ApiV2;

  @GET('/v2/users')
  Future<List<UserDto>> getUsers();
}
```


8.2 Paginación

```
@RestApi(baseUrl: 'https://api.example.com')
abstract class ApiClient {
    factory ApiClient(Dio dio) = _ApiClient;

    @GET('/users')
    Future<HttpResponse<List<UserDto>>> getUsers(
        @Query('page') int page,
        @Query('limit') int limit,
    );
    // HttpResponse permite acceder a headers (X-Total-Count, etc.)
}
```

8.3 Cancelación de Requests

```
class UserCubit extends Cubit<UserState> {
    final ApiClient apiClient;
    CancelToken? _cancelToken;

    UserCubit(this.apiClient);

    Future<void> loadUsers() async {
        _cancelToken?.cancel('Nueva carga');
        _cancelToken = CancelToken();

        try {
            emit>Loading();
            final users = await apiClient.getUsers(cancelToken:
                _cancelToken);
            emit>Loaded(users);
        } on CancelException {
            // Request cancelada intencionalmente
        }
    }

    @override
    Future<void> close() {
        _cancelToken?.cancel('Cubit cerrado');
        return super.close();
    }
}
```

9. Resumen Ejecutivo

1. **Retrofit** elimina el boilerplate de clientes HTTP usando anotaciones
2. **Dio** es el motor HTTP subyacente (interceptors, cancelación, timeouts)
3. **Interceptors** separan concerns: auth, logging, retry, errores
4. **Integración natural** con json_serializable + injectable + freezed
5. **Testing** se simplifica con mocks tipados

6. Código generado es predecible y fácil de debuggear

Recursos Adicionales

- [Retrofit pub.dev](#)
 - [Dio pub.dev](#)
 - [Retrofit Generator](#)
 - [Dio Interceptors Guide](#)
-

Ver también

- [08-HTTP](#) — Conceptos básicos de HTTP y Dio
 - [04-ALMACENAMIENTO-LOCAL](#) — Persistencia local con drift, Hive
-

En el siguiente módulo

→ [08-flutter-gen-assets.md](#) — Flutter Gen: assets, splash, iconos y localización

Nivel Experto

Nivel Experto: Flutter Gen - Assets, Splash, Iconos y Localización

Flutter tiene un sistema de generación de código integrado para assets, localización y configuración visual. Este archivo cubre las 4 herramientas esenciales que todo proyecto profesional usa.

1. flutter: generate: true y Asset Manifest

1.1 El Sistema de Assets de Flutter

Flutter tiene un sistema built-in de generación de código para assets. Se activa con una línea:

```
# pubspec.yaml
flutter:
  generate: true  # Activa la generación automática
```

Esto hace que Flutter genere automáticamente: - Asset manifest (para imágenes, fuentes, etc.) - Soporte para flutter gen-110n (localización)

1.2 Cómo Funciona

```
# pubspec.yaml
flutter:
  generate: true

  assets:
    - assets/images/
    - assets/fonts/
    - assets/icons/
    - assets/data/
```

Con `generate: true`, Flutter genera un `AssetManifest` interno que permite acceder a assets de forma tipada.

1.3 flutter_gen (Terceros)

El paquete `flutter_gen` de la comunidad va un paso más allá: genera clases Dart con referencias tipadas a cada asset.

```
# pubspec.yaml
dev_dependencies:
  flutter_gen_runner: ^5.4.0
```

```
# flutter_gen.yaml (raíz del proyecto)
flutter_gen:
  output: lib/core/gen/
  line_length: 120

  integrations:
    flutter_svg: true
    flare_flutter: true
    rive: true
    lottie: true
```

Uso:

```
// Sin flutter_gen (strings mágicas)
Image.asset('assets/images/logo.png');

// Con flutter_gen (tipado, autocompletado)
Image.asset(Assets.images.logo);
```

1.4 Estructura de Assets Recomendada

```
assets/
├── images/
│   ├── logo.png
│   ├── logo@2x.png
│   └── background.jpg
├── icons/
│   ├── home.svg
│   └── settings.svg
├── fonts/
│   ├── Inter-Regular.ttf
│   └── Inter-Bold.ttf
├── animations/
│   ├── loading.json      # Lottie
│   └── splash.riv        # Rive
└── data/
    └── default_config.json
```

2. flutter_native_splash

2.1 ¿Qué es?

Genera la pantalla de splash nativa (Android/iOS) desde una configuración YAML, sin código manual.

2.2 Instalación y Configuración

```
# pubspec.yaml
dev_dependencies:
  flutter_native_splash: ^2.4.0
```

```
# flutter_native_splash.yaml
flutter_native_splash:
  # Colores
  color: "#FFFFFF"
  background_image: "assets/images/splash_background.png"

  # Logo
  image: "assets/images/splash_logo.png"
  image_dark: "assets/images/splash_logo_dark.png"

  # Modo oscuro
  color_dark: "#121212"
  android_dark: true
  ios_dark: true

  # Branding (texto debajo del logo)
  branding: "Powered by YourCompany"
  branding_color: "#666666"
  branding_bottom: 30

  # Android
  android: true
  android_gravity: center
  android_screen_orientation: portrait

  # iOS
  ios: true # Usa LaunchScreen.storyboard automáticamente
  ios_content_mode: center

  # Web
  web: false

  # Fullscreen
  fullscreen: true
```

2.3 Comandos

```
# Generar splash
dart run flutter_native_splash:create

# Eliminar splash
dart run flutter_native_splash:remove
```

2.4 Integración con Makefile

```
.PHONY: splash

splash:
    @echo "Generando splash screen..."
    @dart run flutter_native_splash:create
```

3. flutter_launcher_icons

3.1 ¿Qué es?

Genera los iconos de launcher para todas las plataformas desde una imagen fuente.

3.2 Configuración

```
# pubspec.yaml
dev_dependencies:
  flutter_launcher_icons: ^0.13.0
```

```
# flutter_launcher_icons.yaml
flutter_launcher_icons:
  android: true
  ios: true
  image_path: "assets/icons/app_icon.png"

  # Android específico
  adaptive_icon_background: "assets/icons/icon_background.png"
  adaptive_icon_foreground: "assets/icons/icon_foreground.png"
  adaptive_icon_foreground_inset: 20
  adaptive_icon_monochrome: "assets/icons/icon_monochrome.png"

  # iOS específico
  remove_alpha_ios: true
  image_path_ios: "assets/icons/app_icon_ios.png"

  # Web
  web:
    generate: true
    image_path: "assets/icons/app_icon.png"
    favicon: true

  # Windows
  windows:
    generate: true
    image_path: "assets/icons/app_icon.png"
    icon_size: 48

  # MacOS
  macos:
    generate: true
    image_path: "assets/icons/app_icon.png"
```

3.3 Comandos

```
# Generar iconos
dart run flutter_launcher_icons

# Con archivo de configuración específico
dart run flutter_launcher_icons -f flutter_launcher_icons.yaml
```

3.4 Múltiples Flavors

```
# flutter_launcher_icons-dev.yaml
flutter_launcher_icons:
  image_path: "assets/icons/app_icon_dev.png"
  android: true
  ios: true

# flutter_launcher_icons-prod.yaml
flutter_launcher_icons:
  image_path: "assets/icons/app_icon_prod.png"
  android: true
  ios: true
```

```
# Generar para cada flavor
dart run flutter_launcher_icons -f flutter_launcher_icons-dev.yaml
dart run flutter_launcher_icons -f flutter_launcher_icons-prod.yaml
```

4. Flutter Gen-L10n (Localización)

4.1 Configuración

```
# l10n.yaml
arb-dir: lib/l10n/arb
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
output-dir: lib/l10n/gen
nullable-getter: false
header: "// dart format off\n// coverage:ignore-file"
```

```
# pubspec.yaml (necesario para que l10n.yaml sea detectado)
flutter:
  generate: true
```

4.2 Archivos ARB

app_en.arb (template):

```

{
  "@@locale": "en",
  "appName": "Rifa Gestion",
  "@appName": {
    "description": "The application name"
  },

  "welcome": "Welcome to {appName}",
  "@welcome": {
    "description": "Welcome message",
    "placeholders": {
      "appName": {
        "type": "String"
      }
    }
  },

  "itemsCount": "{count, plural, =0{No items} one{1 item}  
  other{{count} items}}",
  "@itemsCount": {
    "description": "Item count with pluralization",
    "placeholders": {
      "count": {
        "type": "int"
      }
    }
  },

  "greeting": "{gender, select, male{Hello sir} female{Hello madam}  
  other{Hello}}",
  "@greeting": {
    "description": "Gender-based greeting",
    "placeholders": {
      "gender": {
        "type": "String"
      }
    }
  }
}

```

app_es.arb (traducción):

```

{
  "@@locale": "es",
  "appName": "Rifa Gestion",

  "welcome": "Bienvenido a {appName}",

  "itemsCount": "{count, plural, =0{Sin items} one{1 item}  
  other{{count} items}}",

  "greeting": "{gender, select, male{Hola señor} female{Hola señora}  
  other{Hola}}"
}

```


4.3 Uso en Código

```
import 'package:flutter/material.dart';
import 'package:flutter_gen/gen_l10n/app_localizations.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context)!;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.appName)),
      body: Center(
        child: Column(
          children: [
            Text(l10n.welcome('Rifa Gestion')),
            Text(l10n.itemsCount(5)), // "5 items"
            Text(l10n.greeting('male')), // "Hola señor"
          ],
        ),
      ),
    );
  }
}
```

4.4 Extension para Conveniencia

```
// lib/core/extensions/localization_ext.dart
import 'package:flutter/material.dart';
import 'package:flutter_gen/gen_l10n/app_localizations.dart';

extension AppLocalizationsX on BuildContext {
  AppLocalizations get l10n => AppLocalizations.of(this!);
}

// Uso: context.l10n.appName
```

4.5 Estructura de ARB Files

```
lib/l10n/
├── arb/
│   ├── app_en.arb      # Template (obligatorio)
│   ├── app_es.arb      # Español
│   ├── app_pt.arb      # Portugués
│   └── app_fr.arb      # Francés
├── gen/                # GENERADO (en .gitignore)
│   ├── app_localizations.dart
│   ├── app_localizations_en.dart
│   └── app_localizations_es.dart
└── extensions/
    └── localization_ext.dart
```

4.6 analysis_options.yaml

```
analyzer:
  exclude:
    - lib/l10n/gen/*
```

4.7 .gitignore

```
# Generated l10n files
lib/l10n/gen/
```

5. Integración en el Proyecto Real

En el monorepo, la configuración de localización está activa:

```
# l10n.yaml
arb-dir: lib/l10n/arb
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
output-dir: lib/l10n/gen
nullable-getter: false
header: "// dart format off\n// coverage:ignore-file"
```

Con `flutter: generate: true` en `pubspec.yaml`, el sistema: 1. Lee los archivos `.arb` en `lib/l10n/arb/` 2. Genera clases Dart tipadas en `lib/l10n/gen/` 3. Provee `AppLocalizations.of(context)` con autocompletado 4. Soporta pluralización y género 5. Cambia idioma automáticamente según el locale del dispositivo

6. Flujo de Trabajo Completo

6.1 Setup Inicial

```
# 1. Activar generación de assets
# Agregar a pubspec.yaml:
#   flutter:
#     generate: true

# 2. Configurar l10n
touch l10n.yaml

# 3. Crear estructura de ARB
mkdir -p lib/l10n/arb
touch lib/l10n/arb/app_en.arb

# 4. Agregar flutter_native_splash
flutter pub add dev:flutter_native_splash
touch flutter_native_splash.yaml
dart run flutter_native_splash:create

# 5. Agregar flutter_launcher_icons
flutter pub add dev:flutter_launcher_icons
touch flutter_launcher_icons.yaml
dart run flutter_launcher_icons

# 6. (Opcional) flutter_gen para assets tipados
flutter pub add dev:flutter_gen_runner
touch flutter_gen.yaml
```

6.2 Día a Día

```
# Agregar nuevo texto
# 1. Editar app_en.arb y app_es.arb
# 2. Ejecutar:
flutter gen-l10n
# 3. Usar en código: context.l10n.nuevoTexto
```

7. Resumen Ejecutivo

Herramienta	Archivo Config	Comando	Genera
Assets	pubspec.yaml	flutter build	A
flutter_gen	flutter_gen.yaml	build_runner	C ti
Splash	flutter_native_splash.yaml	dart run flutter_native_splash:create	N s
Iconos	flutter_launcher_icons.yaml	dart run flutter_launcher_icons	A
Localización	l10n.yaml	flutter gen-l10n	C ti

Reglas de oro: 1. `generate: true` activa todo el ecosistema 2. Los ARB files son el estándar de localización 3. Los archivos generados (`gen/`) van en `.gitignore` 4. El splash nativo es configuración, no código

Recursos Adicionales

- [flutter_native_splash pub.dev](#)
 - [flutter_launcher_icons pub.dev](#)
 - [flutter_gen pub.dev](#)
 - [Flutter Internationalization Guide](#)
 - [ARB Format Specification](#)
-

Ver también

- [07-retrofit-api-client.md](#) — Clientes HTTP con Retrofit
 - [15-DISEÑO-RESPONSIVO](#) — Diseño responsivo y adaptativo
-

En el siguiente módulo

→ [09-dart3-pattern-matching-nativo.md](#) — Dart 3 Pattern Matching nativo

Nivel Experto

Nivel Experto: Dart 3 Pattern Matching Nativo

Dart 3 trae pattern matching al lenguaje. No necesitas librerías externas para expresiones switch complejas.

Switch expressions (Dart 3)

```
// Antes (Dart 2)
String descripcion(Animal animal) {
  if (animal is Perro) {
    return 'Perro: ${animal.nombre}';
  } else if (animal is Gato) {
    return 'Gato: ${animal.nombre}';
  } else {
    return 'Desconocido';
  }
}

// Dart 3: Switch expression
String descripcion(Animal animal) => switch (animal) {
  Perro(nombre: final n) => 'Perro: $n',
  Gato(nombre: final n)  => 'Gato: $n',
  _                      => 'Desconocido',
};
```

Pattern matching con guarded patterns

```
// Filtrar por condiciones
String clasificar(int edad) => switch (edad) {
  < 0 => 'Inválido',
  0   => 'Recién nacido',
  < 13 => 'Niño',
  < 18 => 'Adolescente',
  < 65 => 'Adulto',
  _    => 'Mayor',
};
```

Destructuring de lists y records

```
// Destructurar una lista
void procesar(List<int> nums) {
    switch (nums) {
        case [final a, final b]:
            print('Dos elementos: $a, $b');
        case [final a, ...final resto]:
            print('Primero: $a, resto: $resto');
        case []:
            print('Vacía');
    }
}

// Destructurar records
(String nombre, int edad) persona = ('Ana', 25);

switch (persona) {
    case (final n, final e) when e >= 18:
        print('$n es mayor de edad');
    case (final n, final e):
        print('$n es menor de edad');
}
```

Pattern matching con sealed classes

```
sealed class Resultado<T> {
    const Resultado();
}

class Exito<T> extends Resultado<T> {
    final T dato;
    const Exito(this.dato);
}

class Error<T> extends Resultado<T> {
    final String mensaje;
    const Error(this.mensaje);
}

// Usar con pattern matching
String mostrar<T>(Resultado<T> resultado) => switch (resultado) {
    Exito(dato: final d) => 'Éxito: $d',
    Error(mensaje: final m) => 'Error: $m',
};
```

Record patterns en parámetros

```
// Función que retorna record
(int, int) minMax(List<int> nums) {
  return (nums.reduce((a, b) => a < b ? a : b),
    nums.reduce((a, b) => a > b ? a : b));
}

// Deestructurar al recibir
final (min, max) = minMax([3, 1, 4, 1, 5, 9]);
print('Mínimo: $min, Máximo: $max');
```

Ejercicio práctico

```
// Convierte esto a Dart 3 pattern matching:
String descripcion(Object obj) {
  if (obj is String) {
    if (obj.isEmpty) return 'String vacío';
    return 'String: $obj';
  } else if (obj is int) {
    if (obj < 0) return 'Negativo';
    if (obj == 0) return 'Cero';
    return 'Positivo';
  } else if (obj is List) {
    return 'Lista de ${obj.length} elementos';
  }
  return 'Otro tipo';
}

// Tu versión con switch expression aquí:
```

► Ver solución

Ver también

- [01-fpdart-result-pattern.md](#) — fpdart y Programación Funcional
 - [17-CLEAN-ARCHITECTURE](#) — Arquitectura limpia con patrones funcionales
-

En el siguiente módulo

→ [10-debugging-build-runner.md](#) — Debugging de build_runner y code generation

Nivel Experto

Nivel Experto: Debugging build_runner

*build_runner es poderoso pero frustrante cuando falla.
Aquí están los errores más comunes y cómo arreglarlos.*

Los 5 errores más comunes

1. "A conflicting output" o "Generated files conflict"

```
# Error: Conflicting outputs were detected
```

Causa: Archivos generados en ubicación incorrecta o con nombres duplicados.

Solución:

```
# Limpiar y regenerar
dart run build_runner clean
dart run build_runner build --delete-conflicting-outputs
```

2. "Could not find..."

```
# Error: Could not find package:json_serializable
```

Causa: Falta dependencia o no se ejecutó `pub get`.

Solución:

```
flutter pub get
dart run build_runner build
```

3. Error de importación circular

```
// ❌ ERROR: Import circular
// model_a.dart importa model_b.dart
// model_b.dart importa model_a.dart
```

Solución: Extraer tipos comunes a un archivo separado o usar `part` / `part of`.

4. Generación lenta o stuck

```
# build_runner tarda mucho o se cuelga
```

Solución:

```
# Usar build_runner con más memoria
dart run build_runner build --delete-conflicting-outputs

# 0 para desarrollo continuo:
dart run build_runner watch --delete-conflicting-outputs
```

5. "The getter 'X' isn't defined for the class 'Y'"

Causa: El código generado tiene errores porque la fuente tiene errores.

Solución: 1. Arreglar errores en el código fuente primero 2. Ejecutar `dart analyze` para verificar 3. Regenerar

```
dart analyze
dart run build_runner build --delete-conflicting-outputs
```

Comandos esenciales

```
# Build completo
dart run build_runner build --delete-conflicting-outputs

# Watch mode (regenera al cambiar)
dart run build_runner watch --delete-conflicting-outputs

# Limpiar archivos generados
dart run build_runner clean

# Ver qué se va a generar
dart run build_runner build --dry-run
```

Paquetes que usan build_runner

Paquete	Para qué
<code>json_serializable</code>	JSON serialization
<code>freezed</code>	Immutables + equality
<code>injectable</code>	Dependency injection
<code>auto_route</code>	Routing
<code>source_gen</code>	Custom generators

Checklist de debugging

- ☐ ¿flutter pub get se ejecutó?
 - ☐ ¿Hay errores de análisis? (dart analyze)
 - ☐ ¿Hay imports circulares?
 - ☐ ¿Se usa --delete-conflicting-outputs?
 - ☐ ¿Se limpió con clean antes?
-

Ver también

- `05-build-runner-ecosystem.md` — Ecosistema completo de build_runner
 - `06-json-serializable-freezed.md` — json_serializable y Freezed
-

Fin del módulo 06

Has completado el **Nivel Experto** de Flutter Clean Architecture. Los patrones funcionales, la inyección de dependencias automatizada, el ecosistema de code generation y las técnicas de debugging te dan las herramientas para construir aplicaciones robustas y mantenibles.

Volver al índice: README.md